

# Digital Design & Computer Arch.

## Lecture 10: Microarchitecture Fundamentals & Design

Dr. Mohammad Sadrosadati

Prof. Onur Mutlu

ETH Zürich

Spring 2026

20 March 2026

# Agenda for Today & Next Few Lectures

---

- Instruction Set Architectures (ISA): LC-3 and MIPS
- Assembly programming: LC-3 and MIPS
- Microarchitecture (principles & single-cycle uarch)
- Multi-cycle microarchitecture
- Pipelining
- Issues in Pipelining:
  - Control & Data Dependence Handling
  - State Maintenance and Recovery
- Out-of-Order Execution

|                    |
|--------------------|
| Problem            |
| Algorithm          |
| Program/Language   |
| System Software    |
| SW/HW Interface    |
| Micro-architecture |
| Logic              |
| Devices            |
| Electrons          |

# Readings

---

## ■ This week

- Introduction to microarchitecture and single-cycle microarchitecture
  - H&H, Chapter 7.1-7.3
  - P&P, Appendices A and C
- Multi-cycle microarchitecture
  - H&H, Chapter 7.4
  - P&P, Appendices A and C

## ■ Next week

- Pipelining
  - H&H, Chapter 7.5
- Pipelining Issues
  - H&H, Chapter 7.7, 7.8.1-7.8.3

# Recall: Microarchitecture

---

- **Implementation of the ISA** under specific **design constraints and goals**
- Anything done in hardware without exposure to software
  - Pipelining
  - In-order versus out-of-order instruction execution
  - Memory access scheduling policy
  - Speculative execution
  - Superscalar processing (multiple instruction issue?)
  - Clock gating
  - Caching? Levels, size, associativity, replacement policy
  - Prefetching?
  - Voltage/frequency scaling?
  - Error correction?

# Implementing the ISA: Microarchitecture Basics

# Now That We Know What an ISA Is...

---

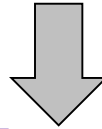
- How do we implement it?
- i.e., **how do we design a system that obeys the hardware/software interface?**
- Aside: “System” can be solely hardware or a combination of hardware and software
  - Recall the “Translation of ISAs”
  - An **ISA** can be converted (by software or hardware) into an **implementation ISA**
- We will assume “completely hardware” implementation for most lectures

# How Does a Machine Process Instructions?

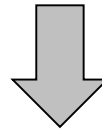
---

- What does processing an instruction mean?
- We will assume the von Neumann model (for now)

AS = Architectural (programmer visible) state before an instruction is processed



**Process instruction**



AS' = Architectural (programmer visible) state after an instruction is processed

- Processing an instruction: Transforming AS to AS' according to the ISA specification of the instruction

# Recall: von Neumann Model/Architecture

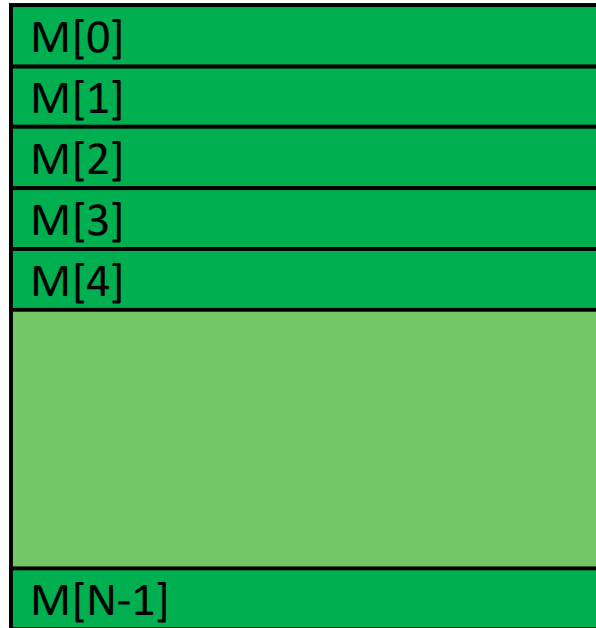
---

**Stored program**

**Sequential instruction processing**

# Recall: Programmer Visible (Architectural) State

---



## Memory

array of storage locations  
indexed by an address



## Registers

- given special names in the ISA (as opposed to addresses)
- general vs. special purpose

## Program Counter

memory address  
of the current (or next) instruction

Instructions (and programs) specify how to transform  
the values of programmer visible state

# The “Process Instruction” Step

---

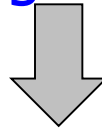
- ISA specifies abstractly what AS' should be, given an instruction and AS
  - It defines an **abstract finite state machine** where
    - State = programmer-visible state
    - Next-state logic = instruction execution specification
  - From ISA point of view, there are no “intermediate states” between AS and AS' during instruction execution
    - One state transition per instruction
- Microarchitecture implements how AS is transformed to AS'
  - There are many choices in implementation
  - We can have programmer-invisible state to optimize the speed of instruction execution: **multiple** state transitions per instruction
    - Choice 1:  $AS \rightarrow AS'$  (transform AS to AS' in a single clock cycle)
    - Choice 2:  $AS \rightarrow AS+MS1 \rightarrow AS+MS2 \rightarrow AS+MS3 \rightarrow AS'$  (take multiple clock cycles to transform AS to AS')

# A Very Basic Instruction Processing Engine

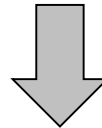
---

- Each instruction takes a single clock cycle to execute
- Only combinational logic is used to implement instruction execution
  - *No intermediate, programmer-invisible state updates*

AS = Architectural (programmer visible) state  
at the beginning of a clock cycle



Process instruction in **one clock cycle**

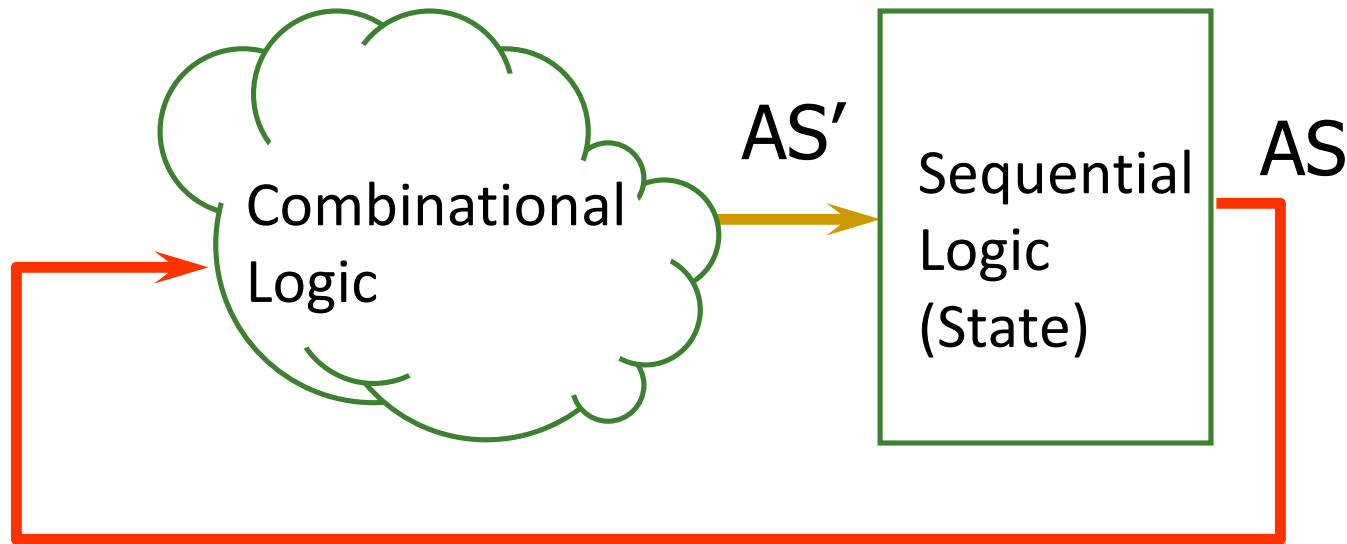


AS' = Architectural (programmer visible) state  
at the end of a clock cycle

# A Very Basic Instruction Processing Engine

---

- Single-cycle machine



- What is the *clock cycle time* determined by?
- What is the *critical path* (i.e., longest delay path) of the combinational logic determined by?

# Single-cycle vs. Multi-cycle Machines

---

## ■ Single-cycle machines

- ❑ Each instruction takes a single clock cycle
- ❑ All state updates made at the end of an instruction's execution
- ❑ Big disadvantage: The slowest instruction determines cycle time → long clock cycle time

## ■ Multi-cycle machines

- ❑ Instruction processing broken into multiple cycles/stages
- ❑ State updates can be made during an instruction's execution
- ❑ Architectural state updates made at the end of an instruction's execution
- ❑ Advantage over single-cycle: The slowest "stage" determines cycle time

- Both single-cycle and multi-cycle machines literally follow the von Neumann model at the microarchitecture level

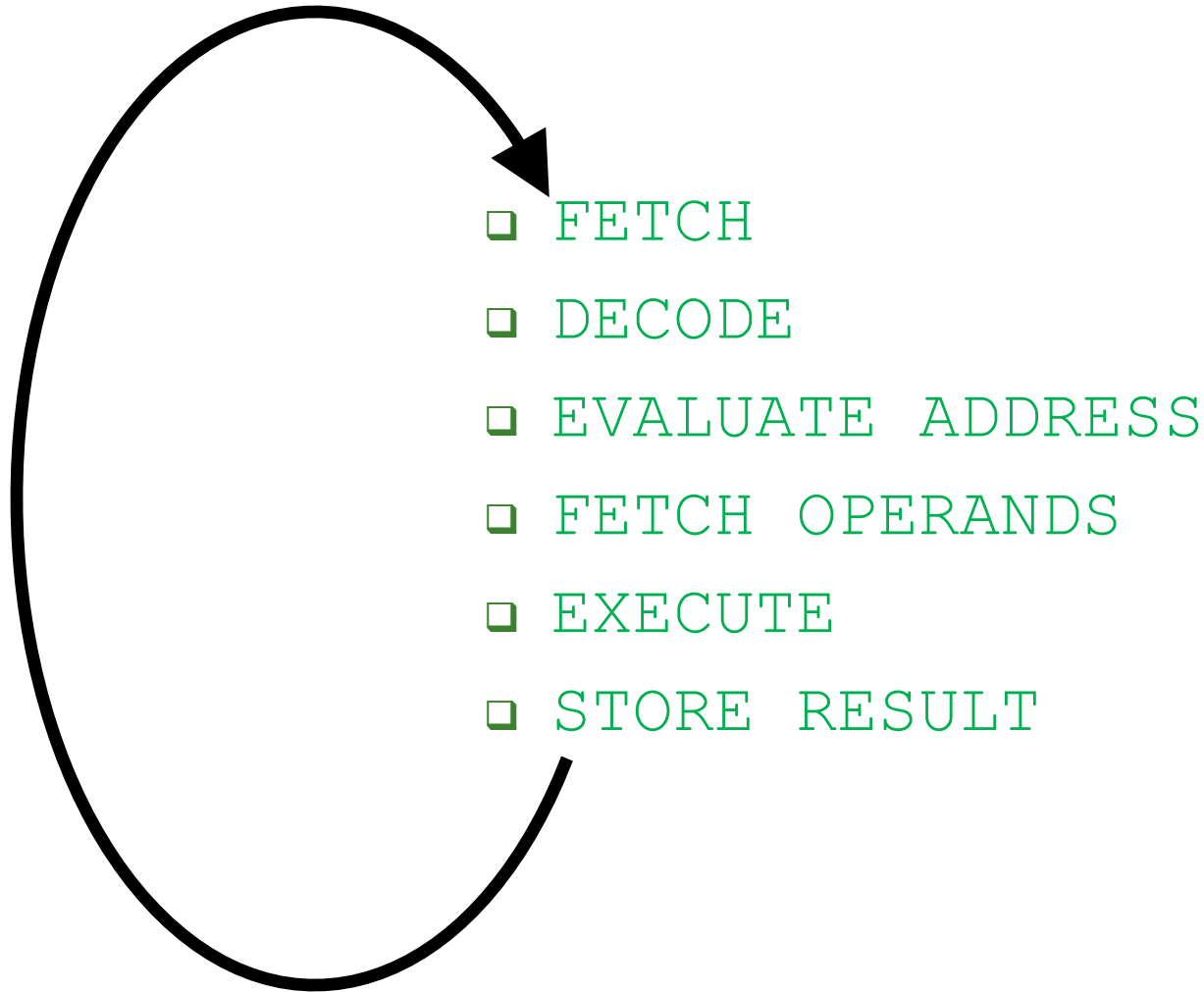
# Instruction Processing “Cycle”

---

- Instructions are processed under the direction of a “control unit” step by step.
- Instruction cycle: Sequence of steps to process an instruction
- Fundamentally, there are six steps:
  - Fetch
  - Decode
  - Evaluate Address
  - Fetch Operands
  - Execute
  - Store Result
- Not all instructions require all six steps (see P&P Ch. 4)

# Recall: The Instruction Processing “Cycle”

---



# Instruction Processing “Cycle” vs. Machine Clock Cycle

---

- Single-cycle machine:
  - All six phases of the instruction processing cycle take a *single machine clock cycle* to complete
- Multi-cycle machine:
  - All six phases of the instruction processing cycle can take *multiple machine clock cycles* to complete
  - In fact, **each phase can take multiple clock cycles to complete**

# Instruction Processing Viewed Another Way

---

- Instructions transform Data (AS) to Data' (AS')
- This transformation is done by functional units
  - Units that “operate” on data
- These units need to be told what to do to the data
- An instruction processing engine consists of two components
  - **Datapath**: Consists of **hardware elements that deal with and transform data signals**
    - **functional units** that operate on data
    - **hardware structures** (e.g., wires, muxes, decoders, tri-state bufs) that enable the flow of data into the functional units and registers
    - **storage units** that store data (e.g., registers)
  - **Control logic**: Consists of **hardware elements that determine control signals, i.e., signals that specify what the datapath elements should do to the data**

# Recall: LC-3: A von Neumann Machine

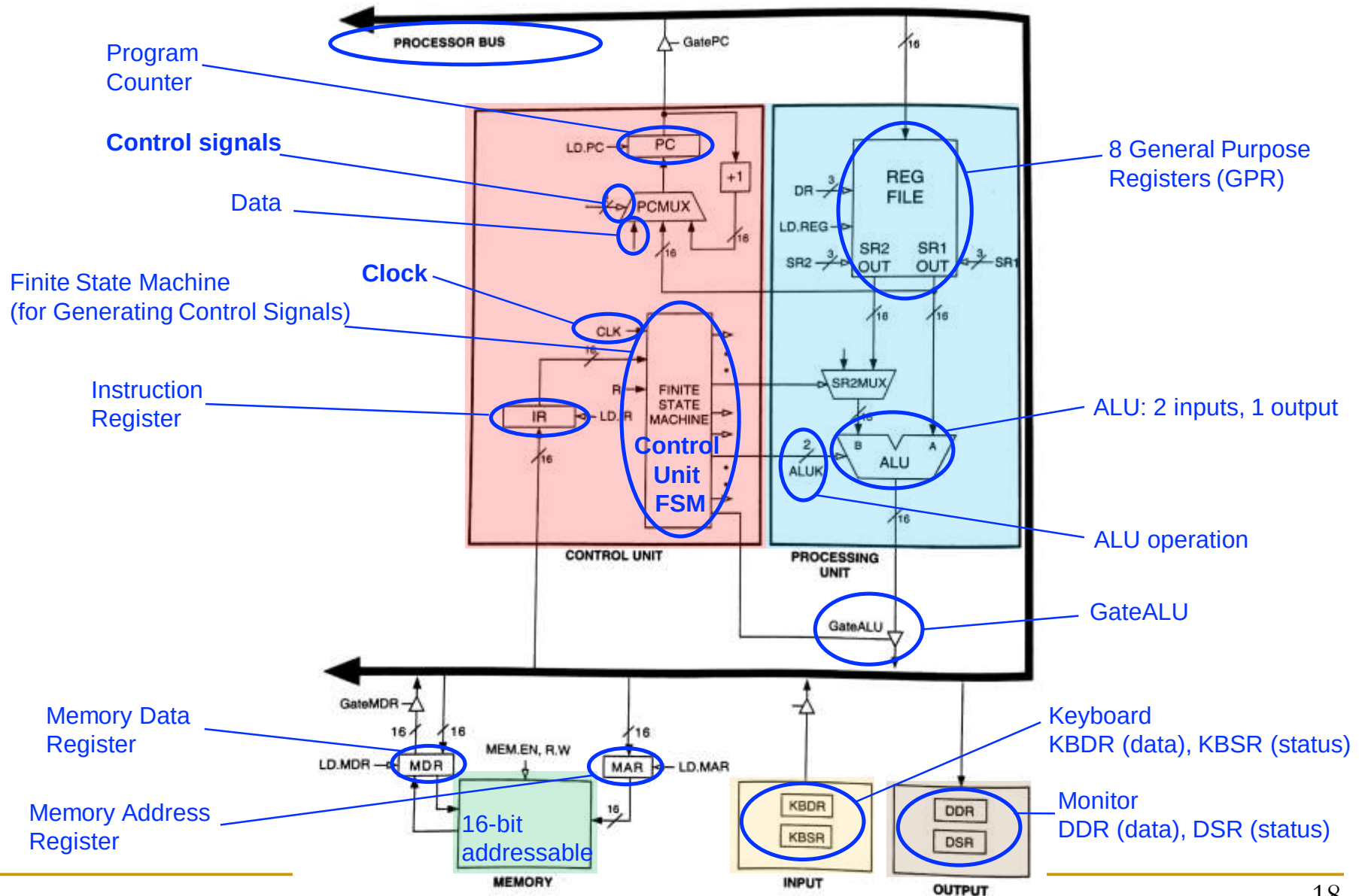


Figure 4.3 The LC-3 as an example of the von Neumann model

# Single-cycle vs. Multi-cycle: Control & Data

---

- Single-cycle machine:
  - Control signals are generated in the same clock cycle as the one during which data signals are operated on
  - Everything related to an instruction happens in one clock cycle (serialized processing)
- Multi-cycle machine:
  - Control signals needed in the next cycle can be generated in the current cycle
  - Latency of control processing can be overlapped with latency of datapath operation (more parallelism)
- See P&P Appendix C for more (microprogrammed multi-cycle microarchitecture)

# Many Ways of Datapath and Control Design

---

- There are many ways of designing the datapath and control logic
- Example ways
  - Single-cycle, multi-cycle, pipelined, out-of-order datapath & control
  - Single-bus vs. multi-bus datapaths
  - Hardwired/combinational vs. microcoded/microprogrammed control
    - Control signals generated by combinational logic versus
    - Control signals stored in a memory structure
- Control signals and structure depend on the datapath design

# Flash-Forward: Performance Analysis

---

- Execution time of a single instruction
  - **{CPI} x {clock cycle time}**      CPI: Cycles Per Instruction
- Execution time of an entire program
  - Sum over all instructions [**{CPI} x {clock cycle time}**]
  - **{# of instructions} x {Average CPI} x {clock cycle time}**
- Single-cycle microarchitecture performance
  - $\text{CPI} = 1$
  - Clock cycle time = long
- Multi-cycle microarchitecture performance
  - CPI = different for each instruction
    - Average CPI  $\rightarrow$  hopefully small
  - Clock cycle time = short

**In multi-cycle, we have two degrees of freedom to optimize independently**

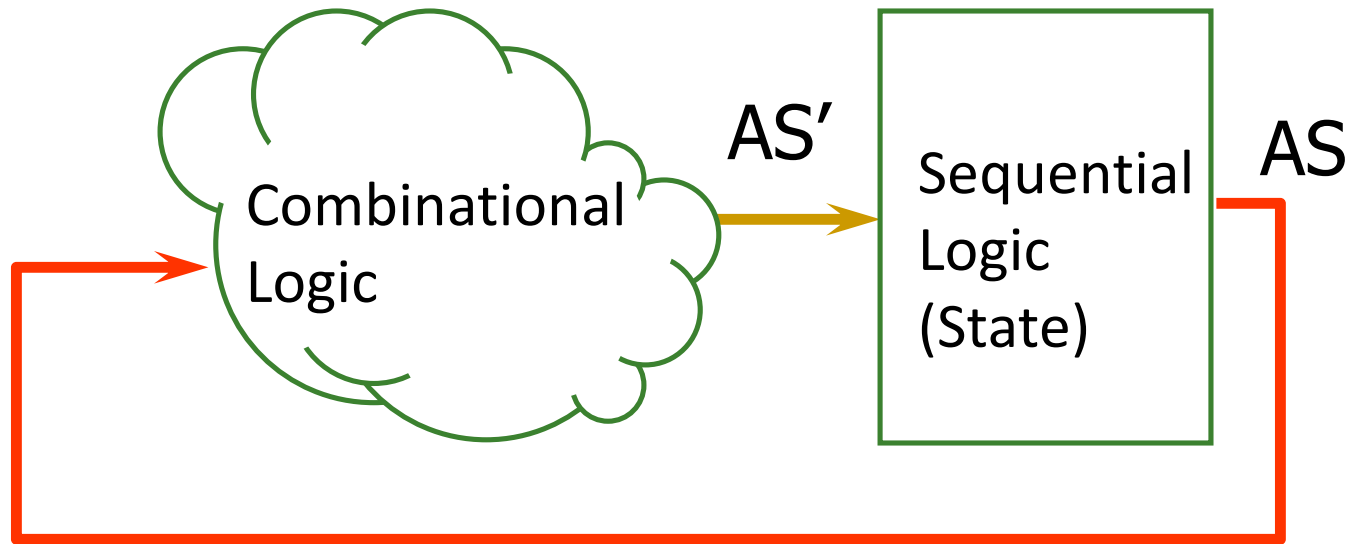
# A Single-Cycle Microarchitecture

## *From the Ground Up*

# Remember...

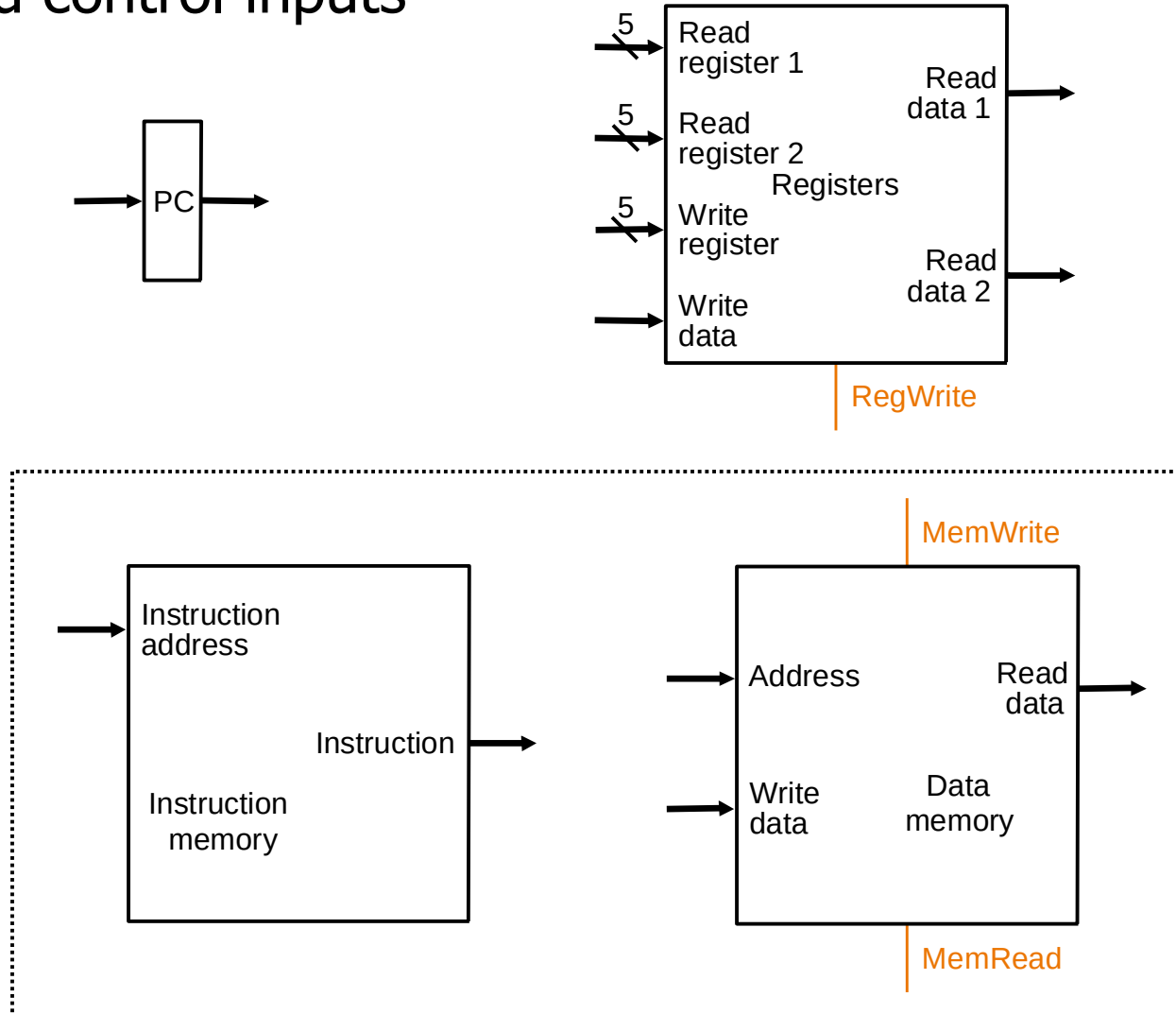
---

- Single-cycle machine

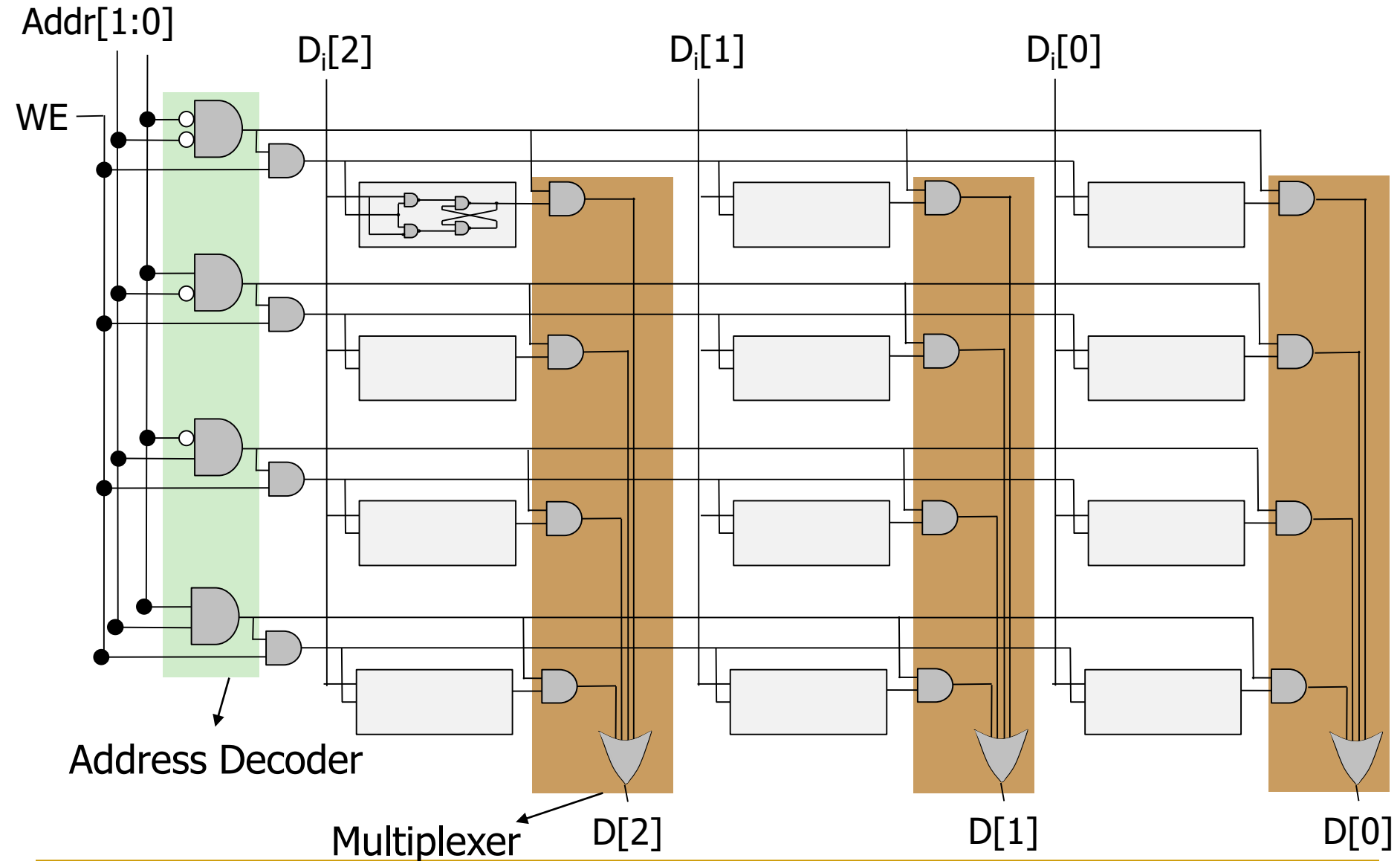


# Let's Start with the State Elements (MIPS)

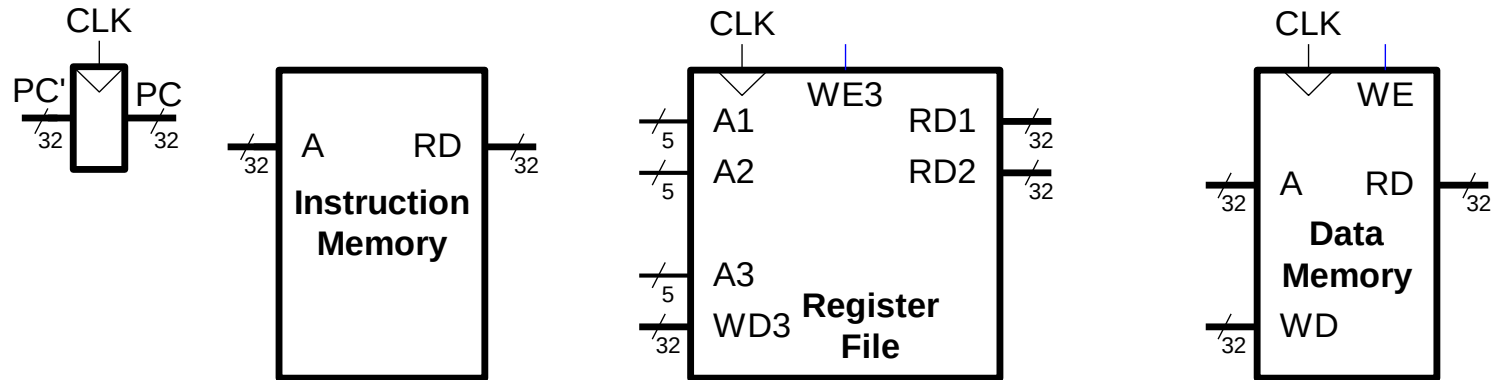
## ■ Data and control inputs



# Recall: A Memory Array (4 locations X 3 bits)



# MIPS State Elements



- ❑ **Program counter:**  
32-bit register
- ❑ **Instruction memory:**  
Takes input 32-bit address A and reads the 32-bit data (i.e., instruction) from that address to the read data output RD
- ❑ **Register file:**  
The 32-element, 32-bit register file has 2 read ports and 1 write port
- ❑ **Data memory:**  
If the write enable, WE, is 1, it writes 32-bit data WD into memory location at 32-bit address A on the rising edge of the clock.  
If the write enable is 0, it reads 32-bit data from address A onto RD.

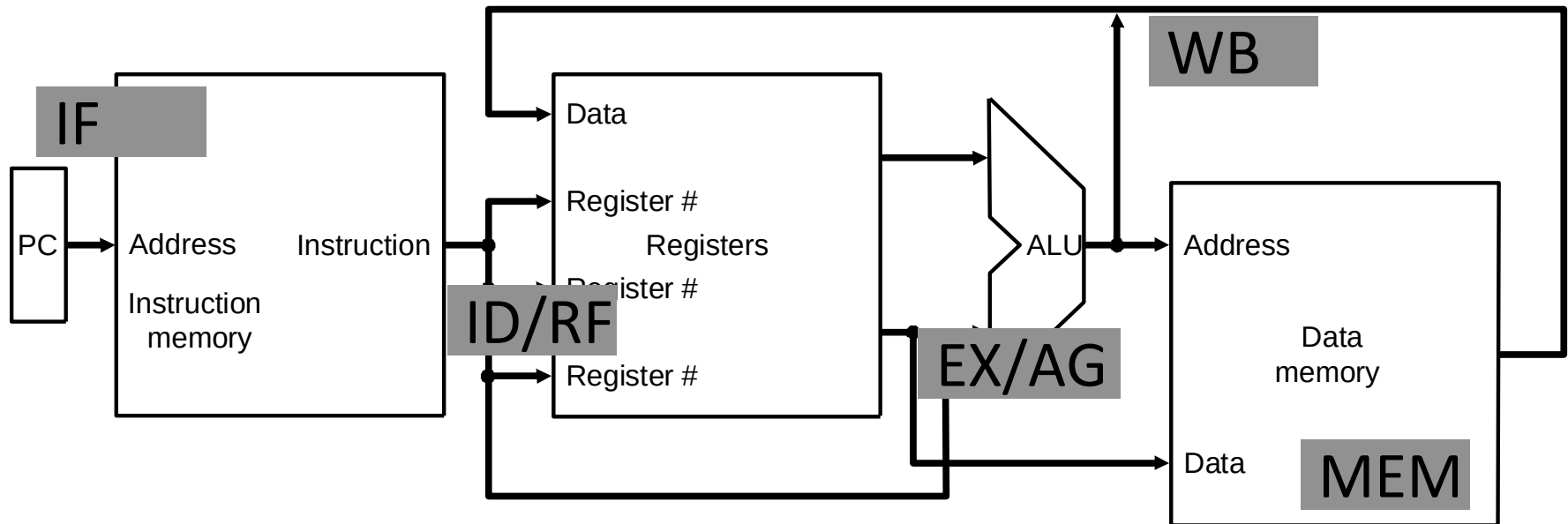
# For Now, We Will Assume

---

- Ultra-fast (i.e., single-cycle) memory and register file
- Combinational read
  - output of the read data port is a combinational function of the memory contents and the corresponding input address
- Synchronous write
  - target location is updated at the positive edge clock transition when the write enable signal is asserted
    - Cannot affect read output in-between positive clock edges
- Single-cycle memory
  - Contrast this with memory that tells when the data is ready
    - i.e., Ready signal: indicating the read or write is done
      - See P&P Appendix C (LC3-b) for multi-cycle memory

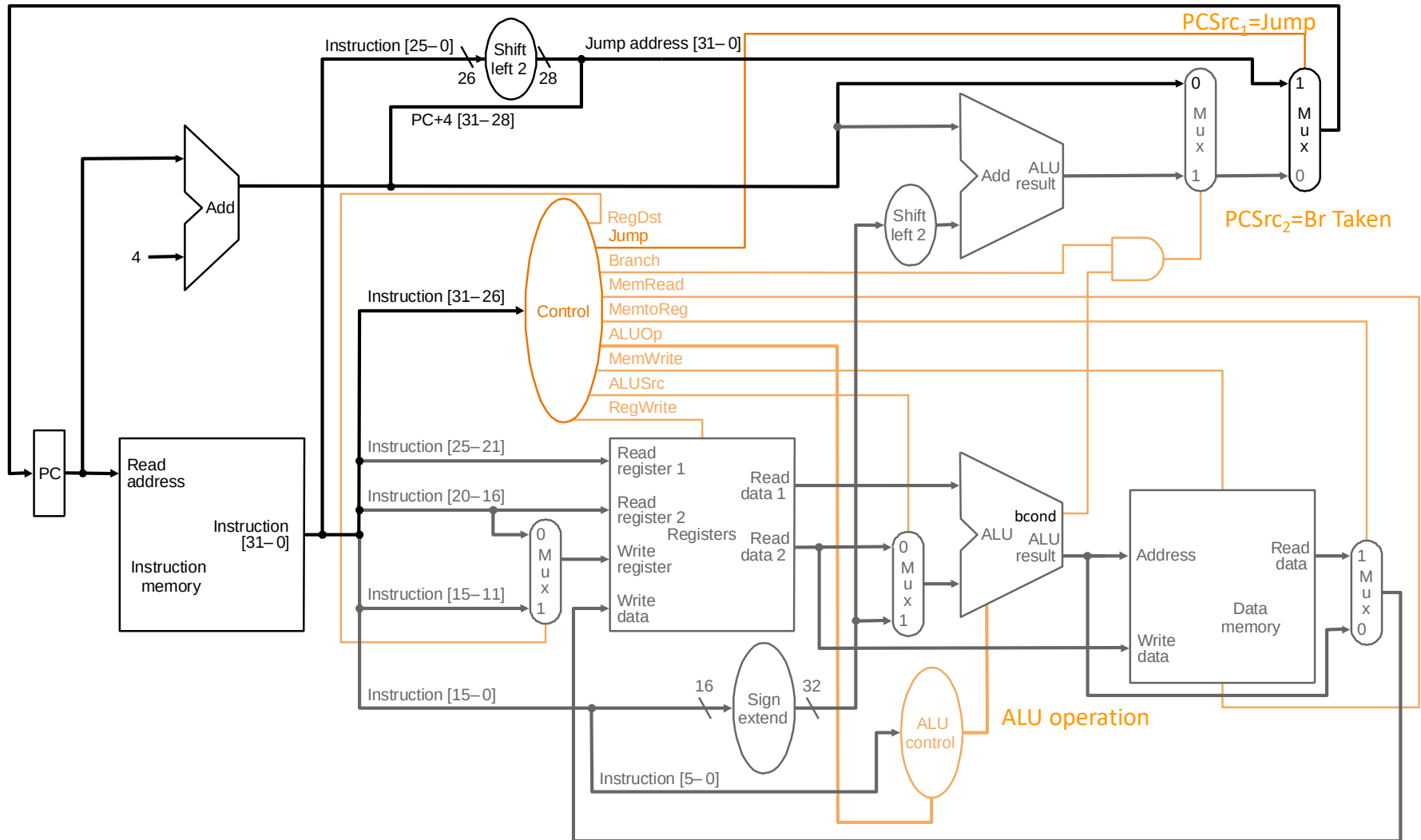
# Instruction Processing

- 5 generic steps
  - ❑ Instruction fetch (IF)
  - ❑ Instruction decode and register operand fetch (ID/RF)
  - ❑ Execute/Evaluate memory address (EX/AG)
  - ❑ Memory operand fetch (MEM)
  - ❑ Store/writeback result (WB)

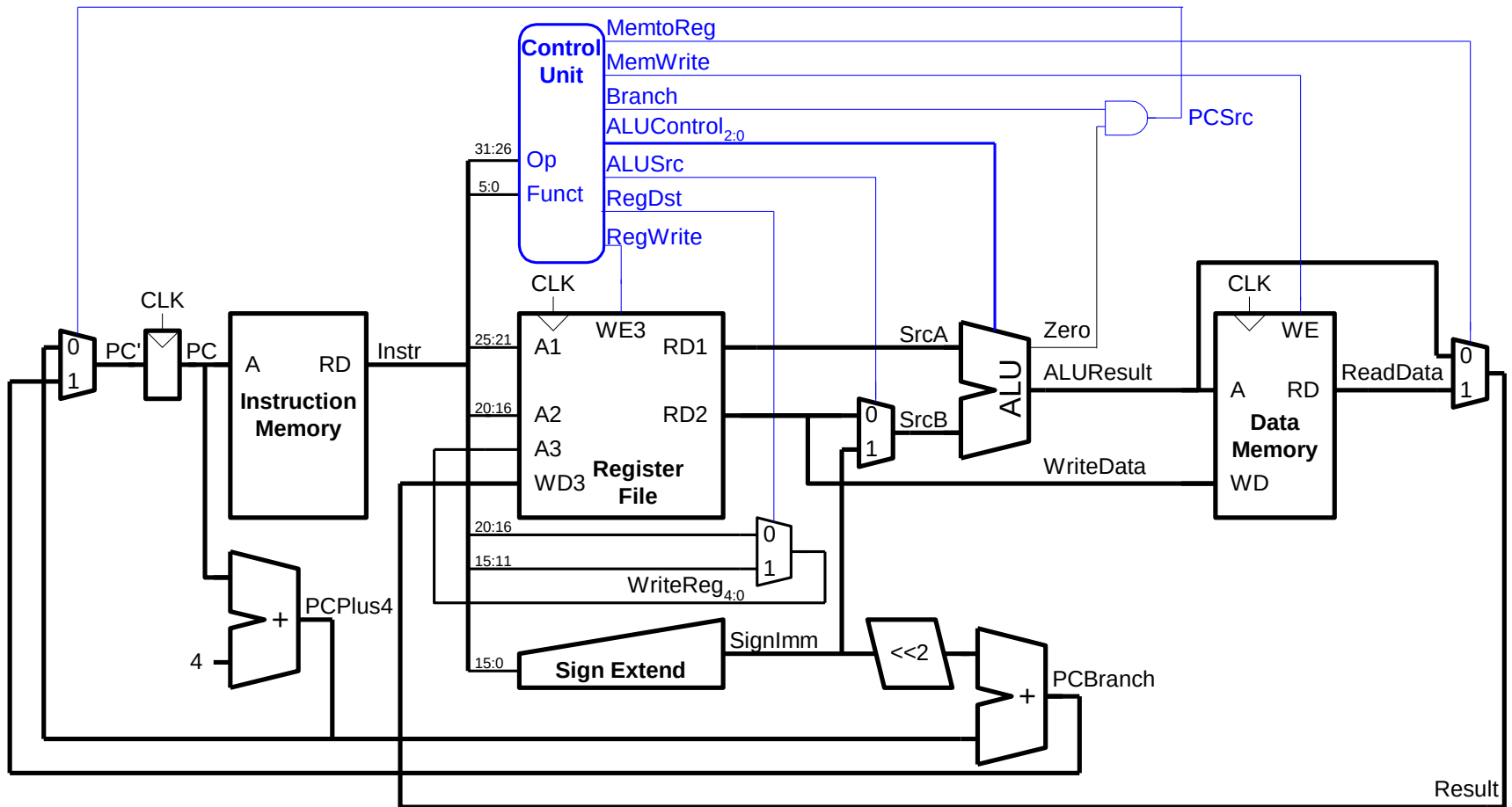


We Need to Provide the  
Datapath & Control Logic  
to Execute All ISA Instructions

# What Is To Come: Single-Cycle MIPS Processor



# Another Complete Single-Cycle Processor



# Single-Cycle Datapath for *Arithmetic and Logical Instructions*

# R-Type ALU Instructions

- R-type: 3 register operands

MIPS assembly (e.g., register-register signed addition)

```
add    $s0, $s1, $s2           # $s0=rd, $s1=rs, $s2=rt
```

Machine Encoding

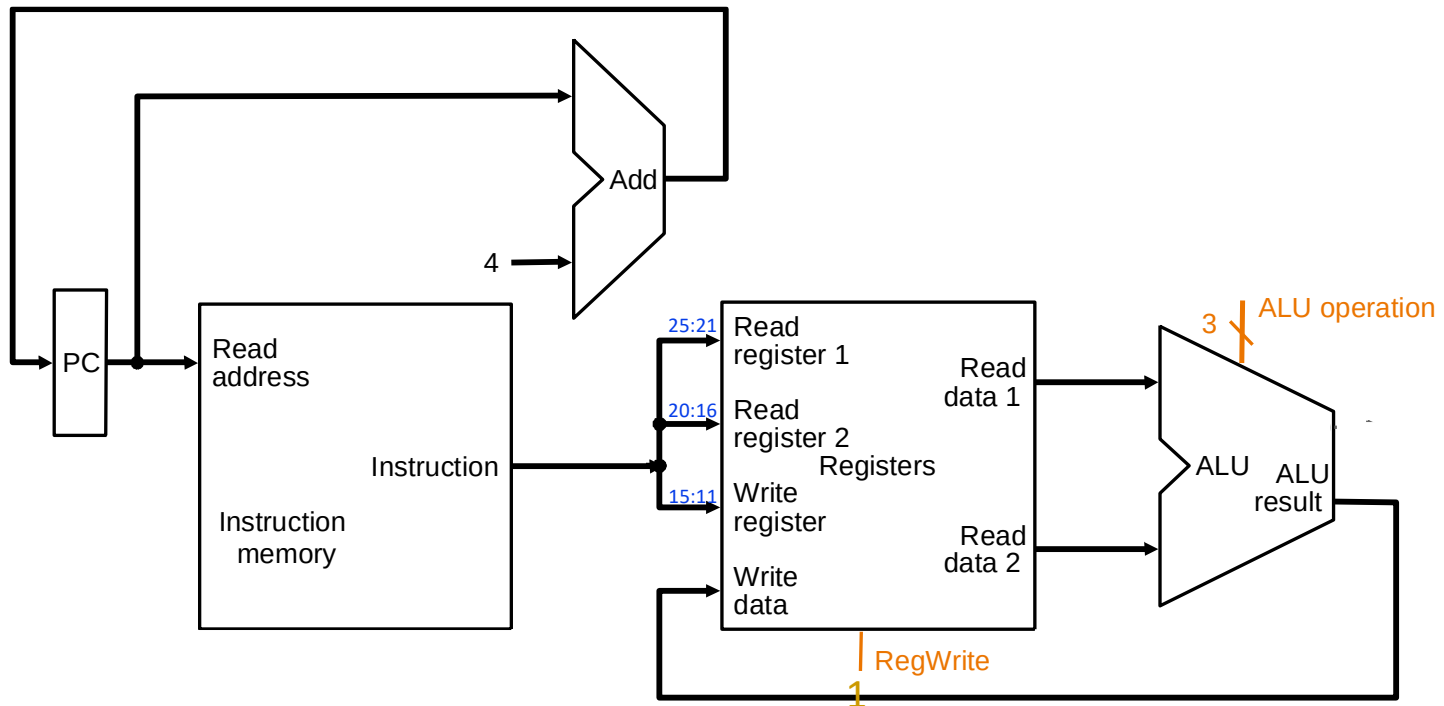
|        |        |        |        |        |          |
|--------|--------|--------|--------|--------|----------|
| 0      | rs     | rt     | rd     | 0      | add (32) |
| 6 bits | 5 bits | 5 bits | 5 bits | 5 bits | 6 bits   |

R-Type

- Semantics

```
if MEM[PC] == add rd rs rt
    GPR[rd] ← GPR[rs] + GPR[rt]
    PC ← PC + 4
```

# (R-Type) ALU Datapath



if MEM[PC] == ADD rd rs rt  
GPR[rd]  $\leftarrow$  GPR[rs] + GPR[rt]  
PC  $\leftarrow$  PC + 4

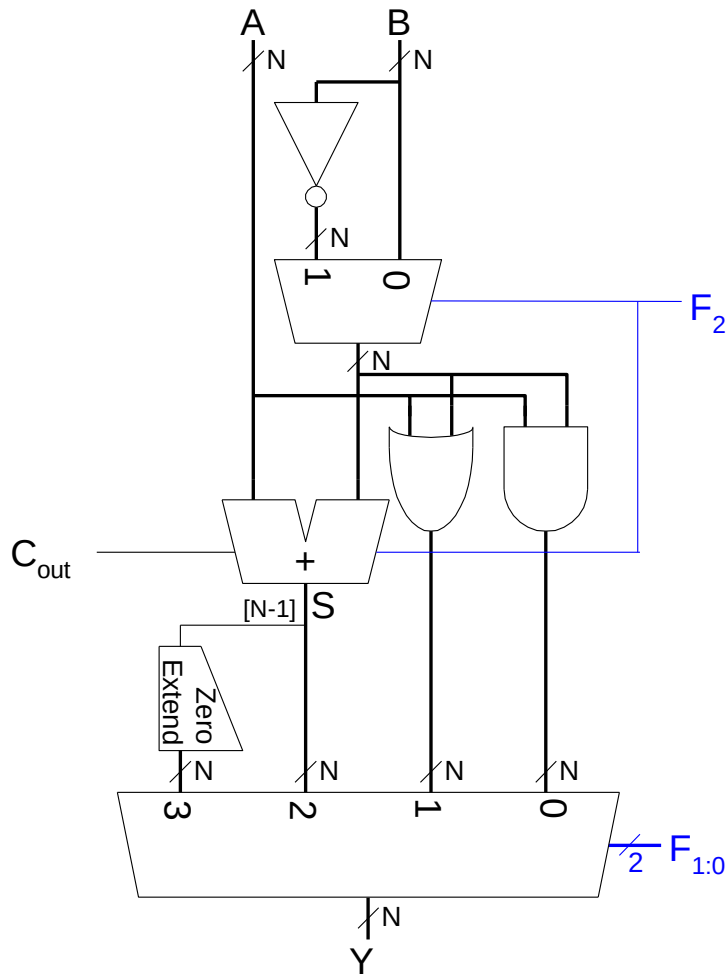


|    |    |    |     |    |
|----|----|----|-----|----|
| IF | ID | EX | MEM | WB |
|----|----|----|-----|----|

Combinational  
state update logic

# Example: ALU Design

- ALU operation ( $F_{2:0}$ ) comes from the control logic



| $F_{2:0}$ | Function        |
|-----------|-----------------|
| 000       | $A \& B$        |
| 001       | $A \mid B$      |
| 010       | $A + B$         |
| 011       | not used        |
| 100       | $A \& \sim B$   |
| 101       | $A \mid \sim B$ |
| 110       | $A - B$         |
| 111       | SLT             |

# I-Type ALU Instructions

- I-type: 2 register operands and 1 immediate

MIPS assembly (e.g., register-immediate signed addition)

```
addi $s0, $s1, 5           # $s0 = rt, $s1 = rs
```

Machine Encoding

|          |        |        |           |
|----------|--------|--------|-----------|
| addi (0) | rs     | rt     | immediate |
| 6 bits   | 5 bits | 5 bits | 16 bits   |

I-Type

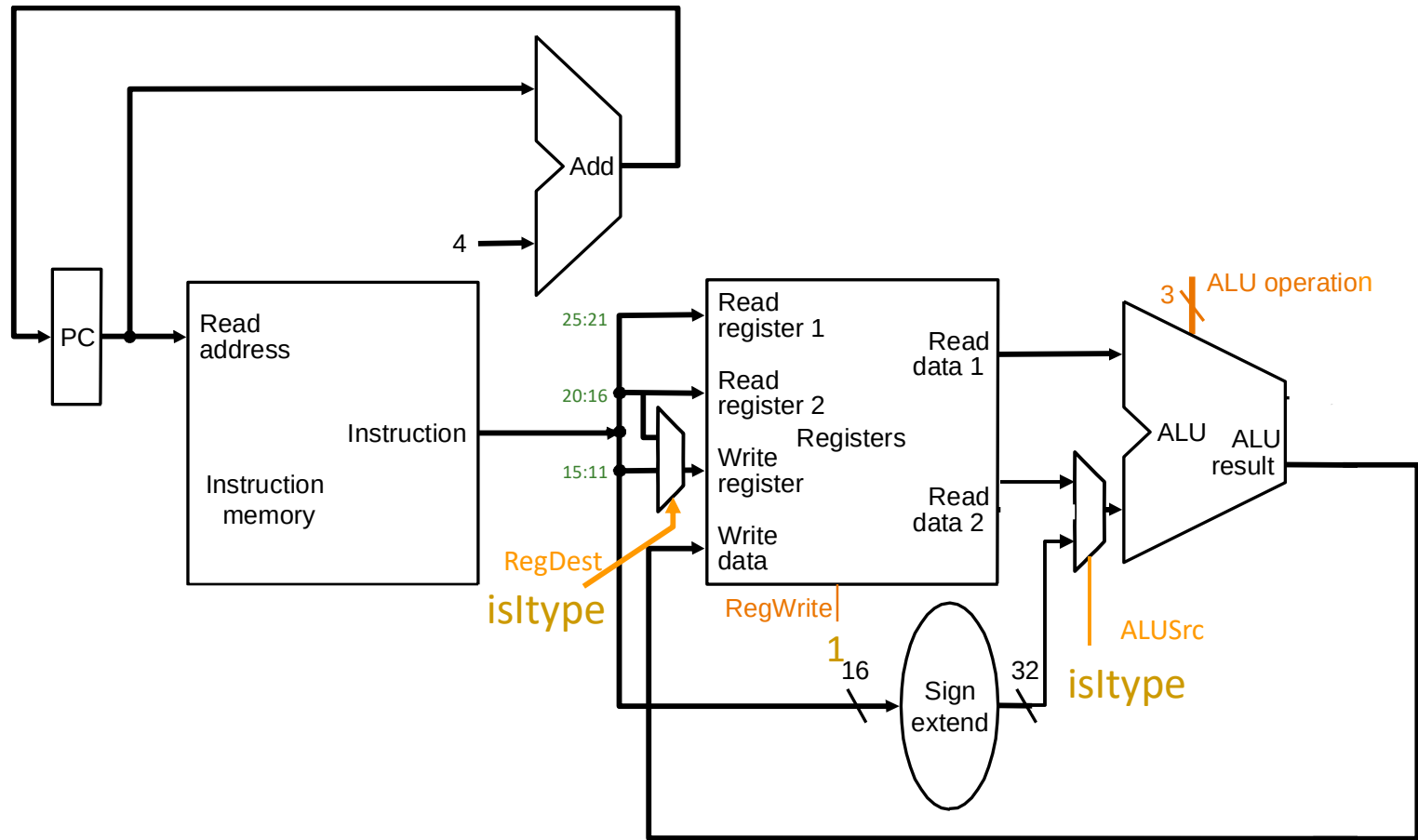
- Semantics

if MEM[PC] == addi rs rt immediate

PC  $\leftarrow$  PC + 4

GPR[rt]  $\leftarrow$  GPR[rs] + sign-extend(immediate)

# Datapath for R- and I-Type ALU Insts.



if MEM[PC] == ADDI rt rs immediate  
 $GPR[rt] \leftarrow GPR[rs] + \text{sign-extend}(\text{immediate})$   
 $PC \leftarrow PC + 4$

Combinational  
 state update logic

# Recall: ADD with one Literal in LC-3

## ■ ADD assembly and machine code

LC-3 assembly

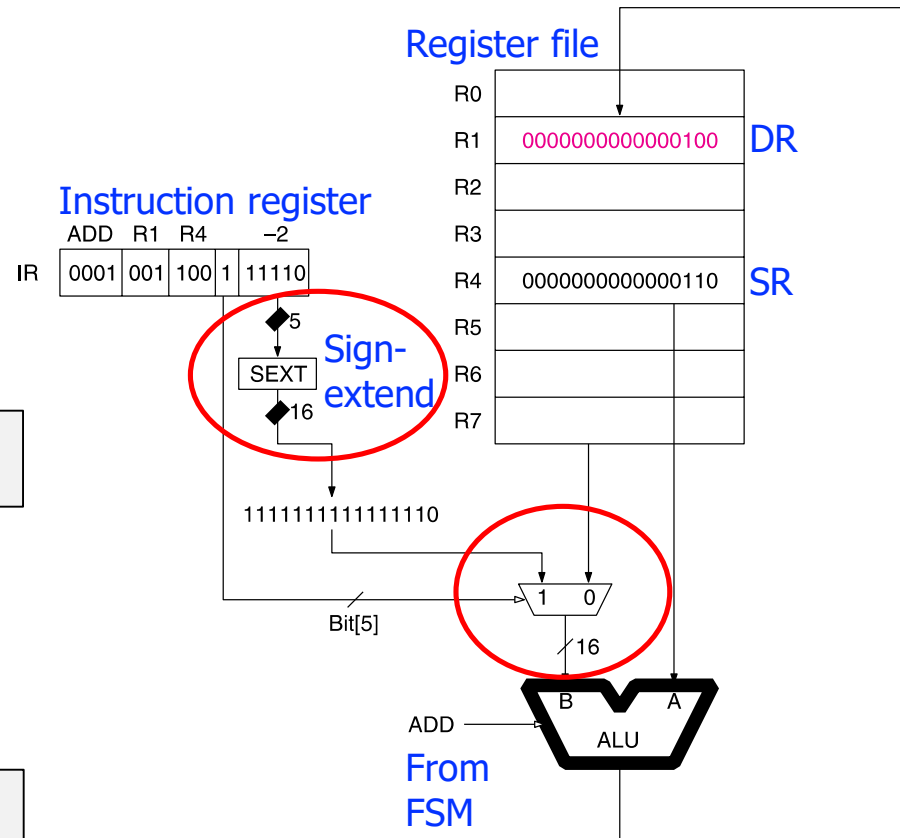
```
ADD R1, R4, #-2
```

Field Values

| OP | DR | SR |   | imm5 |
|----|----|----|---|------|
| 1  | 1  | 4  | 1 | -2   |

Machine Code

| OP   | DR  | SR  |   | imm5  |
|------|-----|-----|---|-------|
| 0001 | 001 | 100 | 1 | 11110 |
| 15   | 12  | 11  | 9 | 8     |
| 6    | 5   | 4   | 3 | 2     |
| 1    | 0   |     |   |       |



# Single-Cycle Datapath for *Data Movement Instructions*

# Load Instructions

## ■ Load 4-byte word

MIPS assembly

```
lw    $s3, 8($s0)           # $s0=rs, $s3=rt
```

Machine Encoding

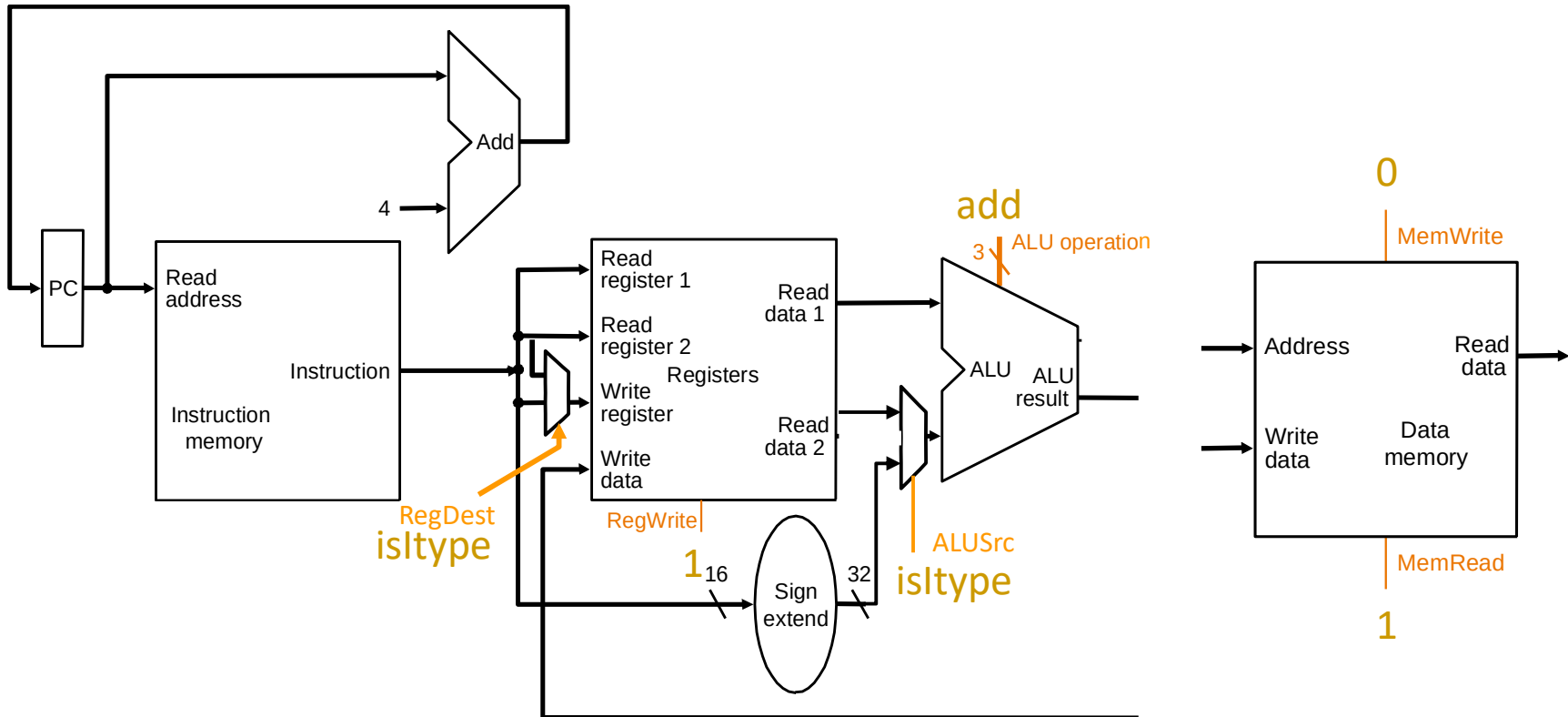


I-Type

## ■ Semantics

if  $\text{MEM}[\text{PC}] == \text{lw rt offset}_{16}(\text{base})$   
     $\text{PC} \leftarrow \text{PC} + 4$   
     $\text{address} = \text{sign-extend}(\text{offset}) + \text{GPR}(\text{base})$   
     $\text{GPR}[\text{rt}] \leftarrow \text{MEM}[\text{address}]$

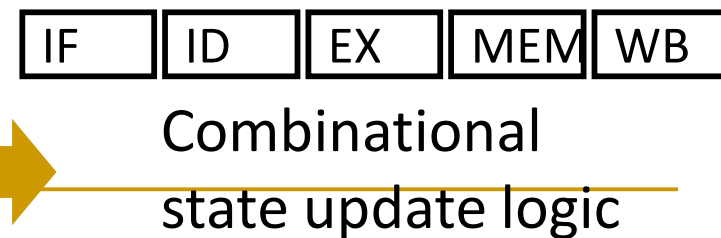
# LW Datapath



```

if MEM[PC]==LW rt offset16 (base)
    address = sign-extend(offset) + GPR[base]
    GPR[rt] ← MEM[address]
    PC ← PC + 4

```



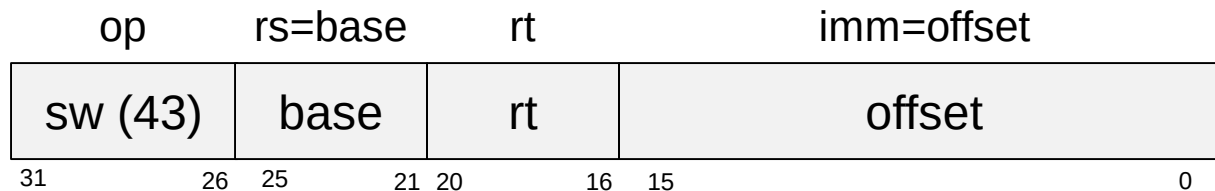
# Store Instructions

- Store 4-byte word

MIPS assembly

```
sw    $s3, 8($s0)           # $s0=rs, $s3=rt
```

Machine Encoding



I-Type

- Semantics

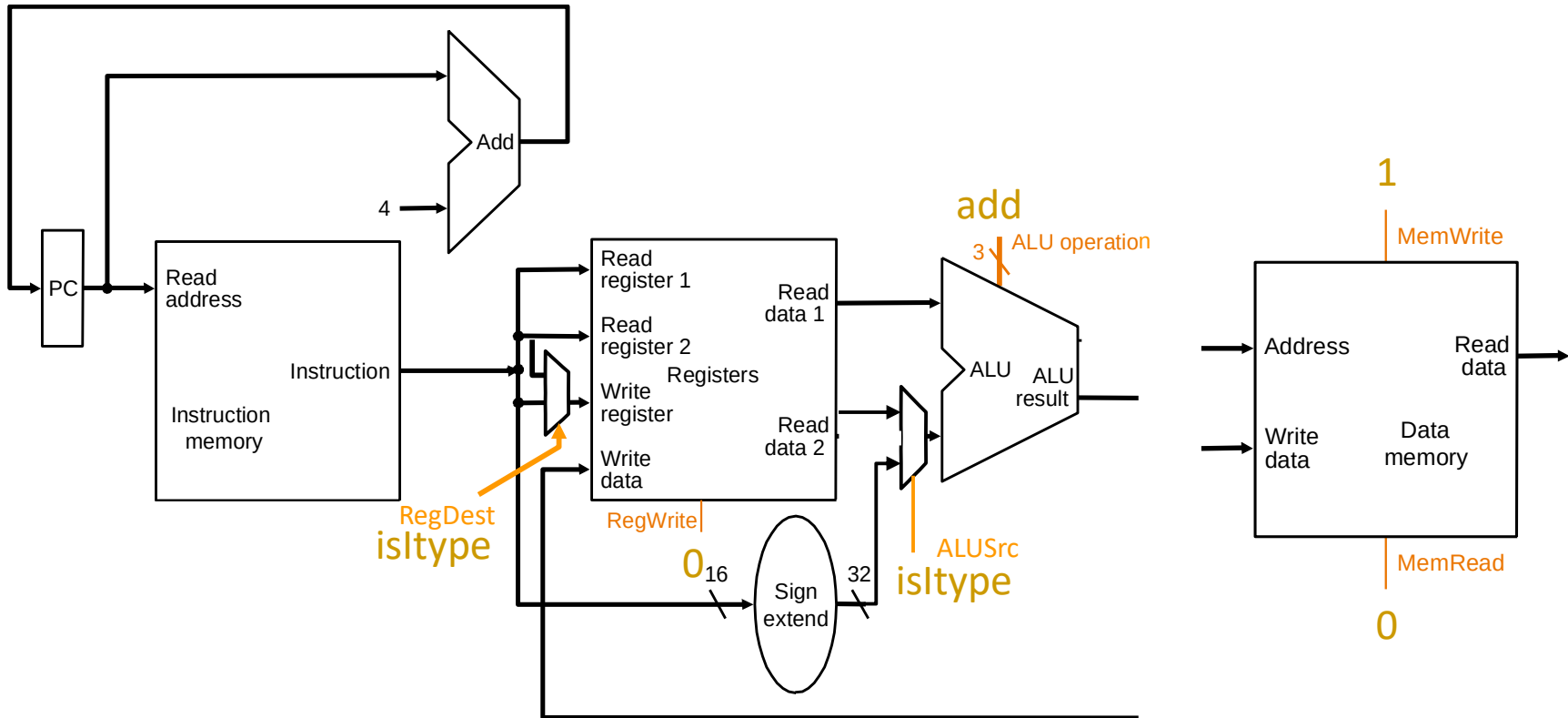
if  $\text{Mem}[\text{PC}] == \text{sw rt offset}_{16}(\text{base})$

$\text{PC} \leftarrow \text{PC} + 4$

$\text{address} = \text{sign-extend}(\text{offset}) + \text{GPR}(\text{base})$

$\text{MEM}[\text{address}] \leftarrow \text{GPR}[\text{rt}]$

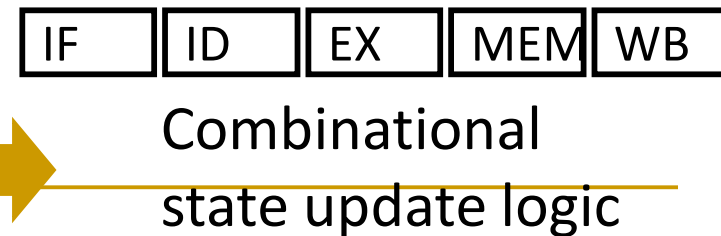
# SW Datapath



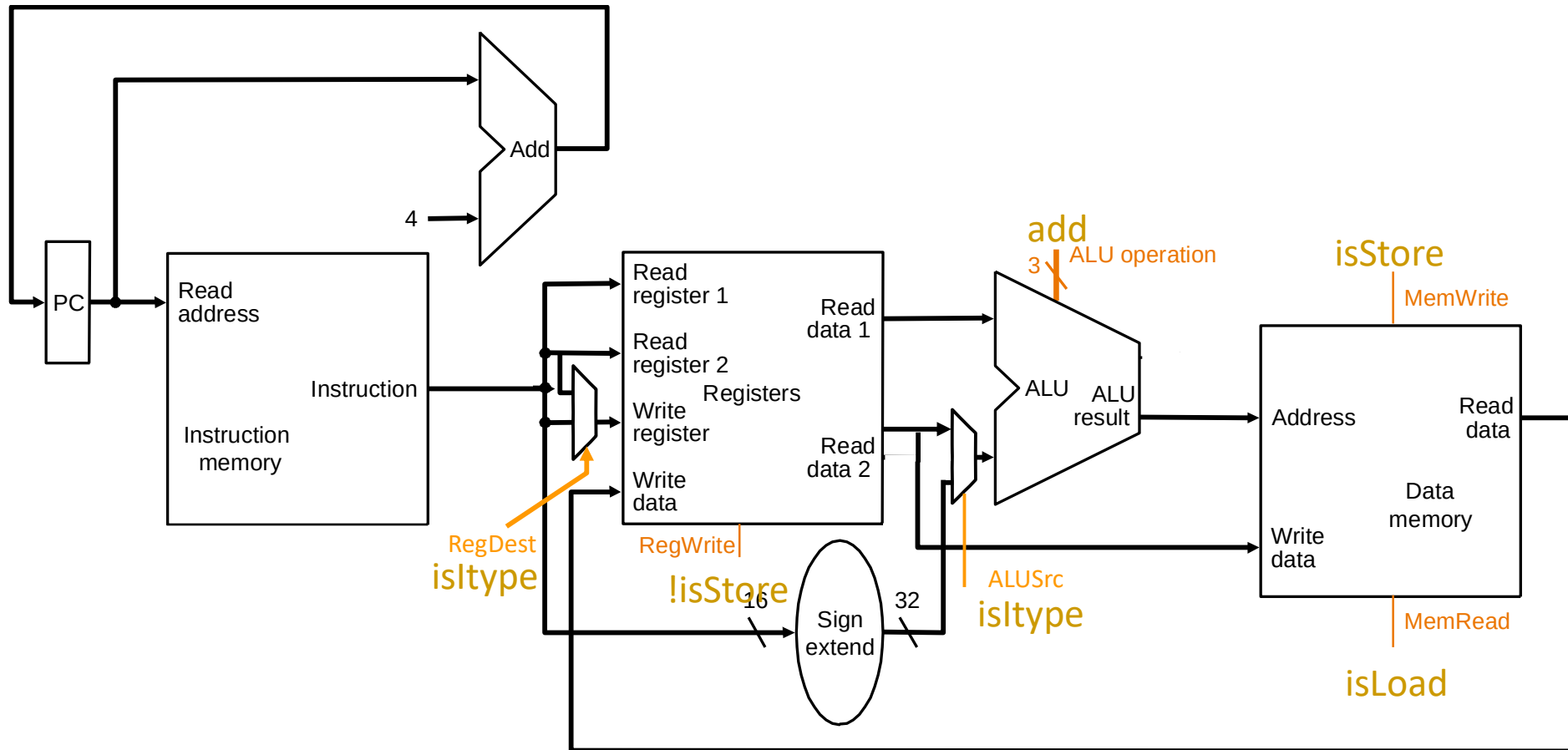
```

if MEM[PC]==SW rt offset16 (base)
    address = sign-extend(offset) + GPR[base]
    MEM[address] ← GPR[rt]
    PC ← PC + 4

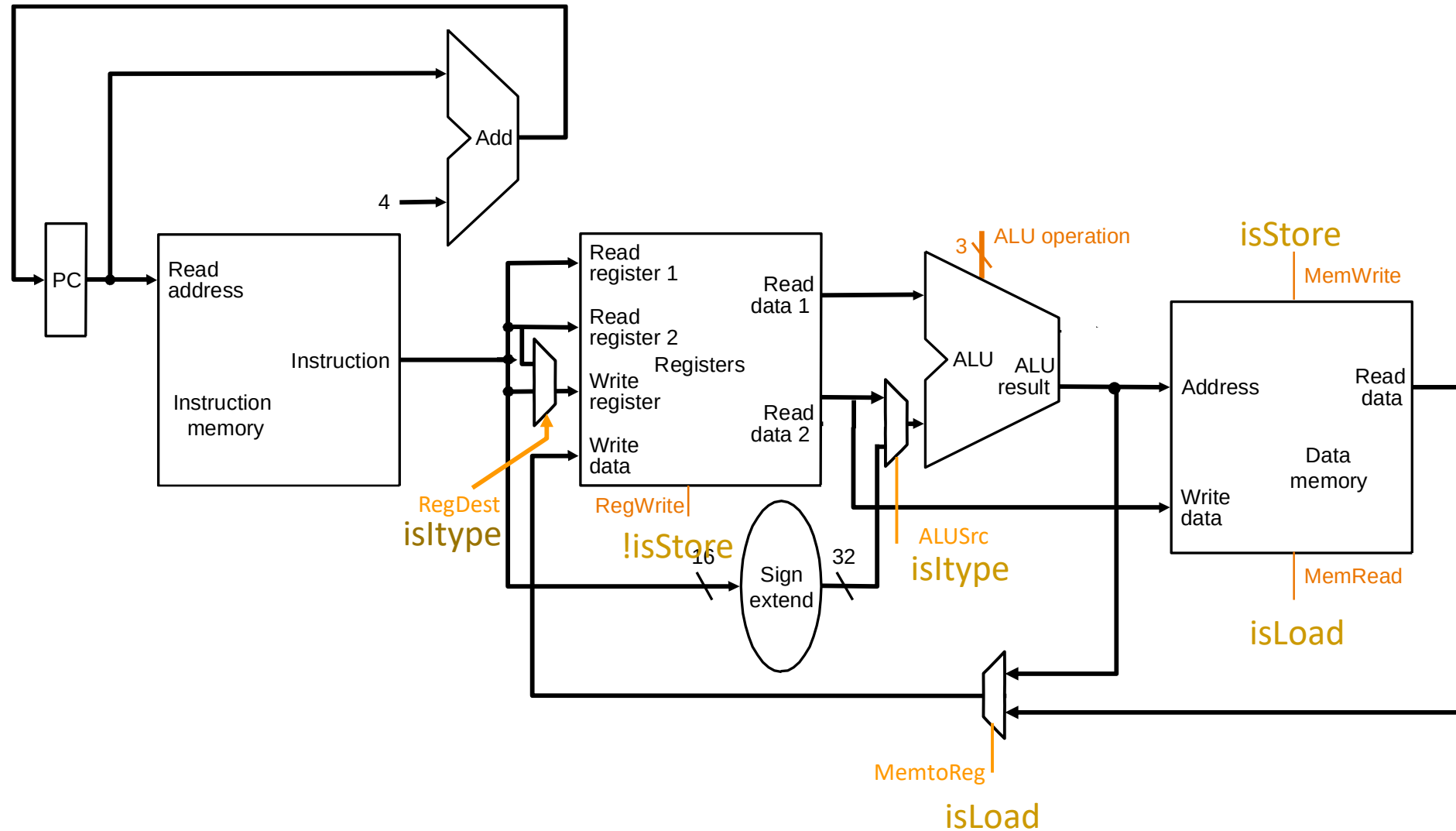
```



# Load-Store Datapath



# Datapath for Non-Control-Flow Insts.

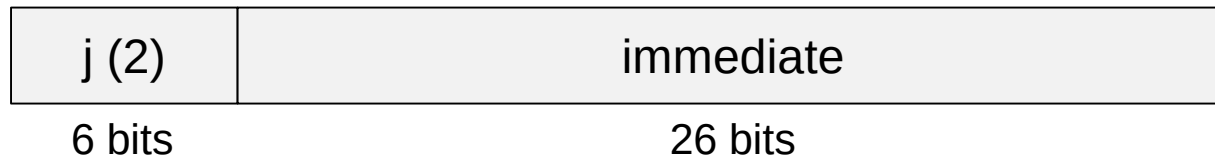


# Single-Cycle Datapath for *Control Flow Instructions*

# Jump Instruction

- Unconditional branch or jump

j    target



J-Type

- 2 = opcode
- immediate (target) = target address

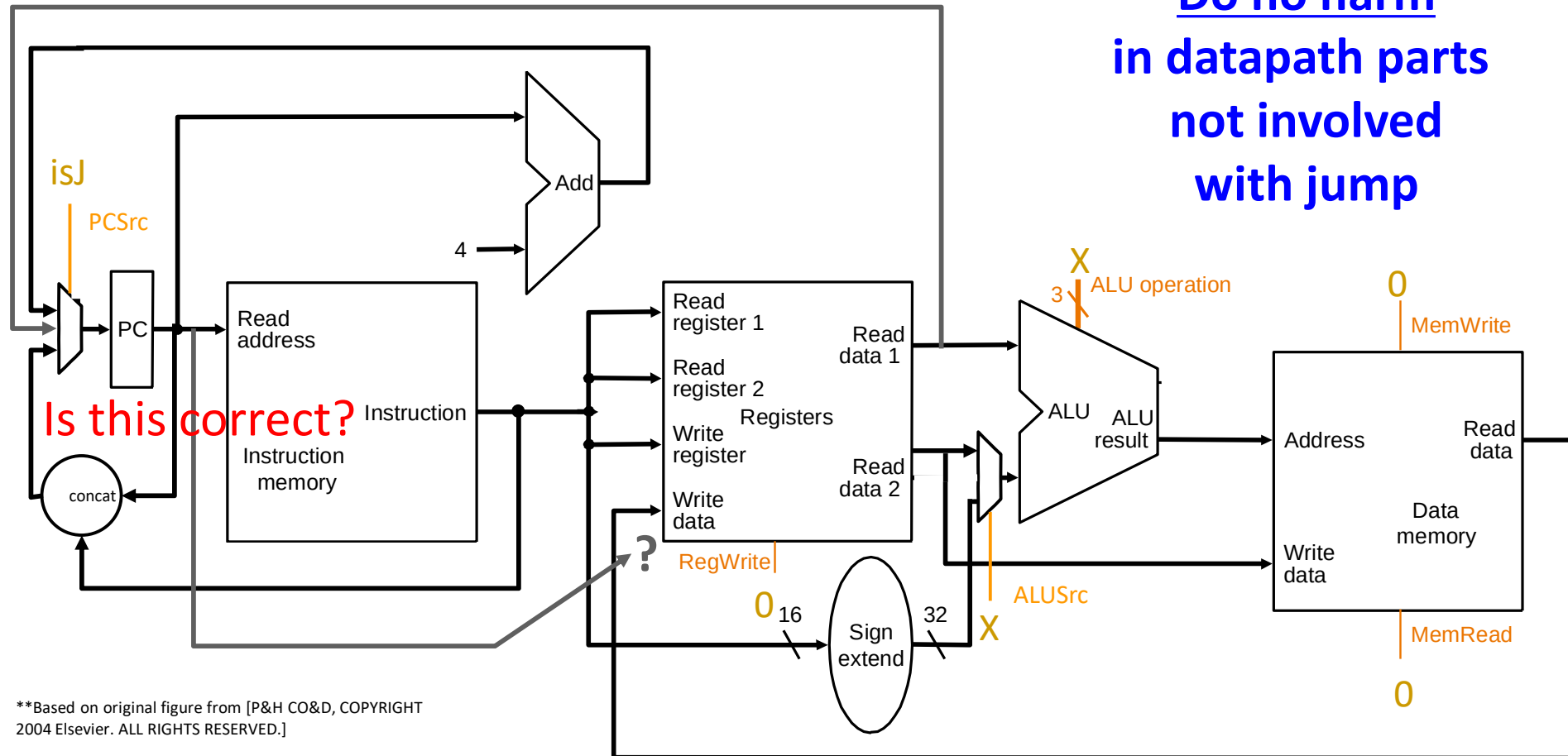
- Semantics

if  $\text{MEM}[\text{PC}] == j \text{ immediate}_{26}$   
     $\text{target} = \{ \text{PC}^\dagger[31:28], \text{immediate}_{26}, 2' \text{ b}00 \}$   
     $\text{PC} \leftarrow \text{target}$

<sup>†</sup> This is the incremented PC

# Unconditional Jump Datapath

Do no harm  
in datapath parts  
not involved  
with jump



if  $\text{MEM}[\text{PC}] == \text{J immediate26}$   
 $\text{PC} = \{ \text{PC}^\dagger[31:28], \text{immediate26}, 2' \text{ b}00 \}$

What about JR, JAL, JALR?

# Other Jumps in MIPS

---

- ❑ **jr: jump register**

Semantics

if  $\text{MEM}[\text{PC}] == \text{jr rs}$   
 $\text{PC} \leftarrow \text{GPR}(\text{rs})$

- ❑ **jal: jump and link (function calls)**

Semantics

if  $\text{MEM}[\text{PC}] == \text{jal immediate}_{26}$   
 $\$ra \leftarrow \text{PC} + 4$   
 $\text{target} = \{ \text{PC}^\dagger[31:28], \text{immediate}_{26}, 2'b00 \}$   
 $\text{PC} \leftarrow \text{target}$

- ❑ **jalr: jump and link register**

Semantics

if  $\text{MEM}[\text{PC}] == \text{jalr rs}$   
 $\$ra \leftarrow \text{PC} + 4$   
 $\text{PC} \leftarrow \text{GPR}(\text{rs})$

---

<sup>†</sup> This is the incremented PC

# Aside: MIPS Cheat Sheet

- [https://safari.ethz.ch/digitaltechnik/spring2023/lib/exe/fetch.php?media=mips\\_reference\\_data.pdf](https://safari.ethz.ch/digitaltechnik/spring2023/lib/exe/fetch.php?media=mips_reference_data.pdf)

- On the course website

## MIPS Reference Data

### CORE INSTRUCTION SET

| NAME                   | MINI-MNEMONIC                                                           | OPERATION (in Words) | OPCODE (FUNCTION) |
|------------------------|-------------------------------------------------------------------------|----------------------|-------------------|
| Add                    | $R[Rd] \leftarrow R[Rs] + R[Rt]$                                        | (1) 0 730            |                   |
| Add Immediate          | $R[Rd] \leftarrow R[Rs] + \text{SignExt}(imm)$                          | (12) 0 30            |                   |
| Add Imm. Unsigned      | $R[Rd] \leftarrow R[Rs] + \text{SignExt}(imm)$                          | (2) 30               |                   |
| ADD Unsigned           | $R[Rd] \leftarrow R[Rs] + R[Rt]$                                        | (1) 230              |                   |
| And                    | $R[Rd] \leftarrow R[Rs] \& R[Rt]$                                       | (1) 240              |                   |
| And Immediate          | $R[Rd] \leftarrow R[Rs] \& \text{ZeroExt}(imm)$                         | (13) 40              |                   |
| Branch On Equal        | $\text{if } (R[Rs] == R[Rt]) \text{ goto } PC + 4 + \text{BranchAdd}$   | (4) 30               |                   |
| Branch On Not Equal    | $\text{if } (R[Rs] \neq R[Rt]) \text{ goto } PC + 4 + \text{BranchAdd}$ | (4) 50               |                   |
| Jump                   | $\text{goto } PC + 4 + \text{BranchAdd}$                                | (1) 70               |                   |
| Jump and Link          | $R[Rt] \leftarrow PC + 4 + \text{BranchAdd}$                            | (1) 70               |                   |
| Jump Register          | $R[Rt] \leftarrow R[Rs]$                                                | (1) 080              |                   |
| Load Byte Unsigned     | $R[Rd] \leftarrow \text{M}[Rs + \text{imm}]$                            | (24) 30              |                   |
| Load Halfword Unsigned | $R[Rd] \leftarrow \text{M}[Rs + \text{imm}]$                            | (24) 30              |                   |
| Load Word              | $R[Rd] \leftarrow \text{M}[Rs + \text{imm}]$                            | (24) 30              |                   |
| Load Upper Word        | $R[Rd] \leftarrow \text{M}[Rs + \text{imm}]$                            | (24) 30              |                   |
| Store                  | $\text{M}[Rs + \text{imm}] \leftarrow R[Rs]$                            | (24) 30              |                   |
| Store Byte             | $\text{M}[Rs + \text{imm}] \leftarrow R[Rs]$                            | (24) 30              |                   |
| Store Halfword         | $\text{M}[Rs + \text{imm}] \leftarrow R[Rs]$                            | (24) 30              |                   |
| Store Word             | $\text{M}[Rs + \text{imm}] \leftarrow R[Rs]$                            | (24) 30              |                   |
| Subtract               | $R[Rd] \leftarrow R[Rs] - R[Rt]$                                        | (1) 0 730            |                   |
| Subtract Unsigned      | $R[Rd] \leftarrow R[Rs] - R[Rt]$                                        | (1) 230              |                   |

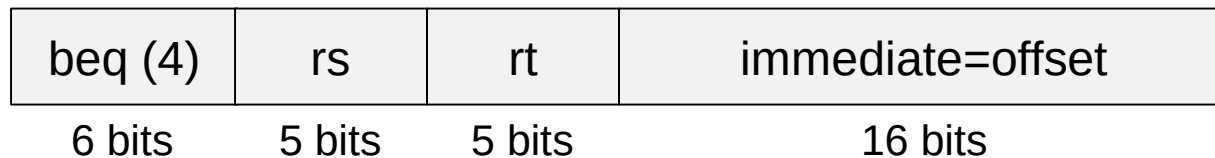
(1) May cause overflow exception.  
(2) SignExt(imm) = 16(imm) <> 0 (imm, immediate).  
(3) ZeroExt(imm) = 16(imm), immediate.  
(4) BranchAdd = 16(imm) <> 0 (imm, immediate, 256).  
(5) BranchAdd = 16(imm) <> 0 (imm, immediate, 256).  
(6) Operands considered unsigned numbers (i.e., 2's comp.).  
(7) Always round up; R[Rs] = 1 if any value, 0 if not zero.

| BASIC INSTRUCTION FORMATS |                                                                   |         |    |           |       |      |
|---------------------------|-------------------------------------------------------------------|---------|----|-----------|-------|------|
| 0                         | opcode                                                            | rs      | rt | rd        | shamt | func |
| 1                         | opcode <th>rs</th> <th>rt</th> <td colspan="3">immediate</td>     | rs      | rt | immediate |       |      |
| 2                         | opcode <td colspan="2">rs, rt</td> <td colspan="3">rd, shamt</td> | rs, rt  |    | rd, shamt |       |      |
| 3                         | opcode                                                            | address |    |           |       |      |

# Conditional Branch Instructions

## ■ beq (Branch if Equal)

beq    \$s0, \$s1, offset                      # \$s0=**rs**, \$s1=**rt**



I-Type

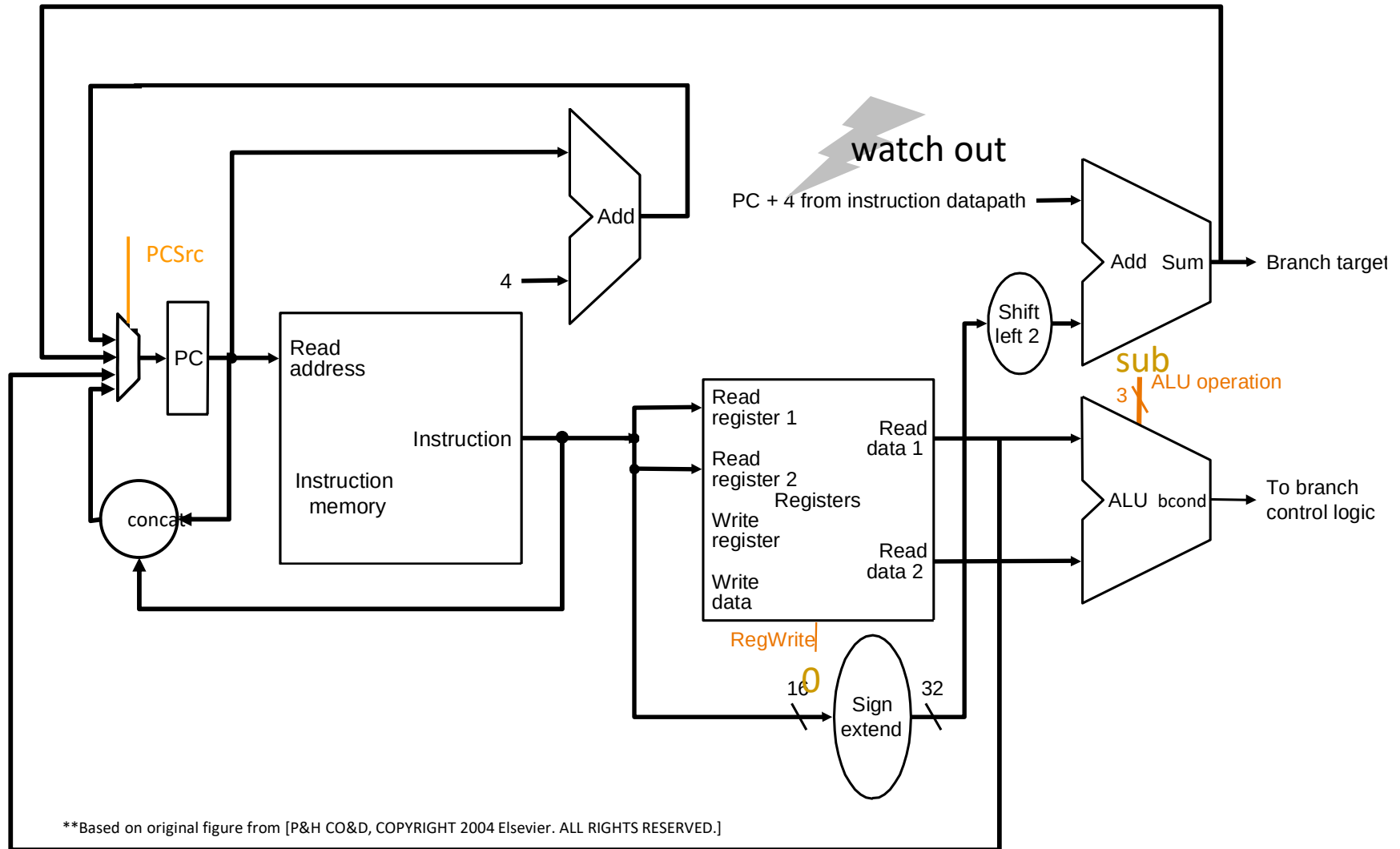
## ■ Semantics (assuming no branch delay slot)

if  $\text{MEM}[\text{PC}] == \text{beq } rs \text{ } rt \text{ } \text{immediate}_{16}$   
     $\text{target} = \text{PC}^{\dagger} + \text{sign-extend}(\text{immediate}) \times 4$   
    if  $\text{GPR}[rs] == \text{GPR}[rt]$  then  $\text{PC} \leftarrow \text{target}$   
    else  $\text{PC} \leftarrow \text{PC} + 4$

□ Variations: beq, bne, blez, bgtz

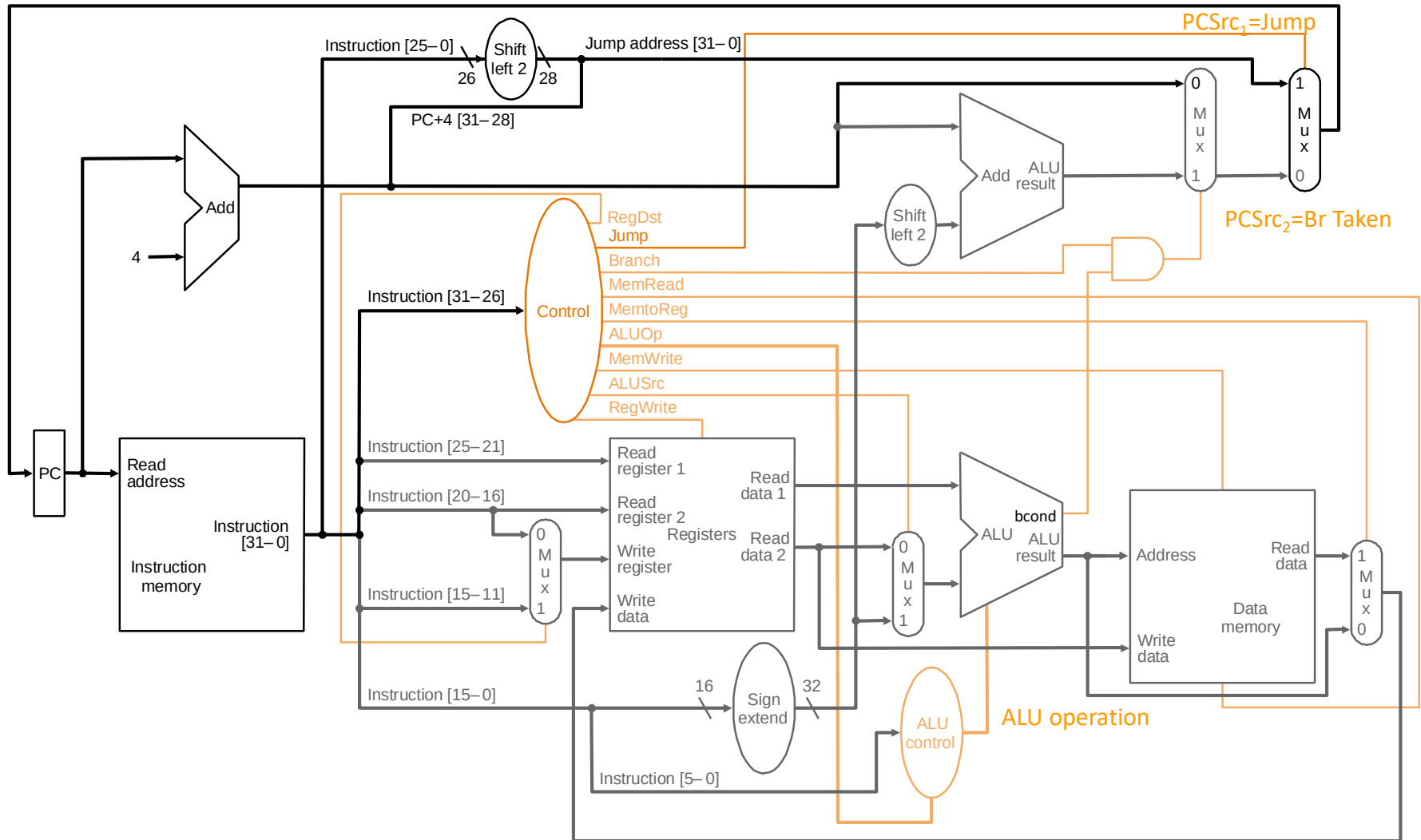
<sup>†</sup> This is the incremented PC

# Conditional Branch Datapath (for you to finish)



Foreshadowing: How to uphold the delayed branch semantics?

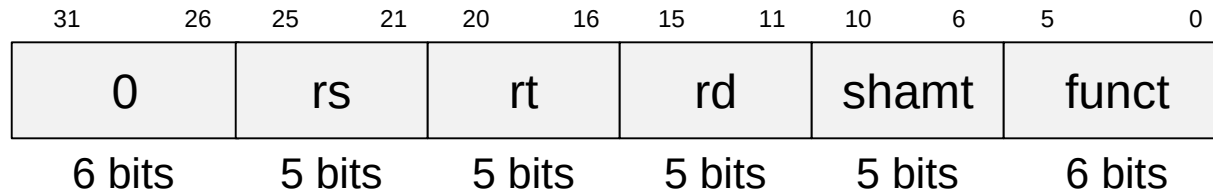
# Putting It All Together



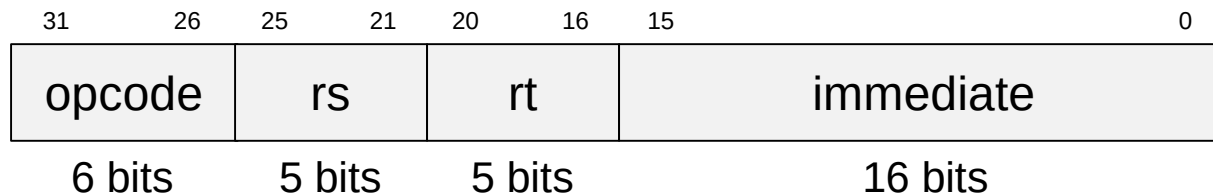
# Single-Cycle Control Logic

# Single-Cycle Hardwired Control

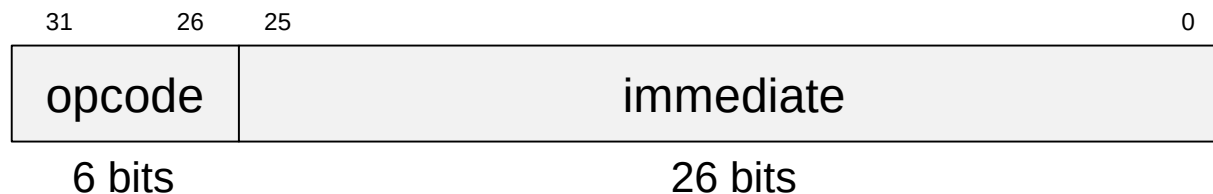
## ■ As combinational function of $\text{Inst} = \text{MEM}[\text{PC}]$



R-Type



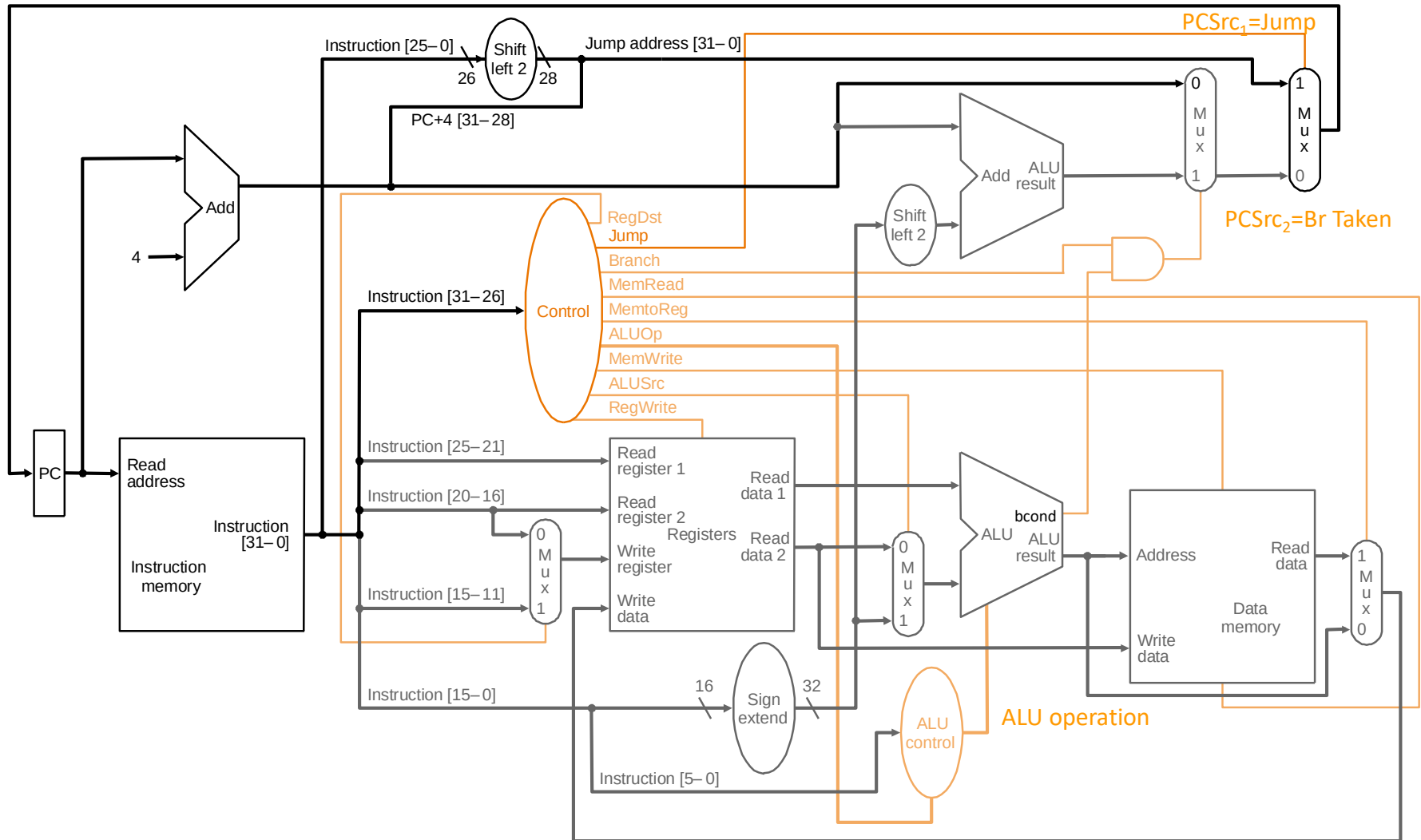
I-Type



J-Type

- Consider
  - ❑ All R-type and I-type ALU instructions
  - ❑ lw and sw
  - ❑ beq, bne, blez, bgtz
  - ❑ j, jr, jal, jalr omitted

# Generate Control Signals (in Orange Color)



# Single-Bit Control Signals (I)

---

|          | When De-asserted                                                    | When asserted                                                        | Equation                                                                                                                                           |
|----------|---------------------------------------------------------------------|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| RegDst   | GPR write select according to <b>rt</b> , i.e., inst[20:16]         | GPR write select according to <b>rd</b> , i.e., inst[15:11]          | <code>opcode==0</code>                                                                                                                             |
| ALUSrc   | 2 <sup>nd</sup> ALU input from 2 <sup>nd</sup> <b>GPR</b> read port | 2 <sup>nd</sup> ALU input from sign-extended 16-bit <b>immediate</b> | <code>(opcode!=0) &amp;&amp;</code><br><code>(opcode!=BEQ) &amp;&amp;</code><br><code>(opcode!=BNE)</code>                                         |
| MemtoReg | Steer <b>ALU result</b> to GPR write port                           | Steer <b>memory output</b> to GPR write port                         | <code>opcode==LW</code>                                                                                                                            |
| RegWrite | GPR <b>write disabled</b>                                           | GPR <b>write enabled</b>                                             | <code>(opcode!=SW) &amp;&amp;</code><br><code>(opcode!=Bxx) &amp;&amp;</code><br><code>(opcode!=J) &amp;&amp;</code><br><code>(opcode!=JR))</code> |

---

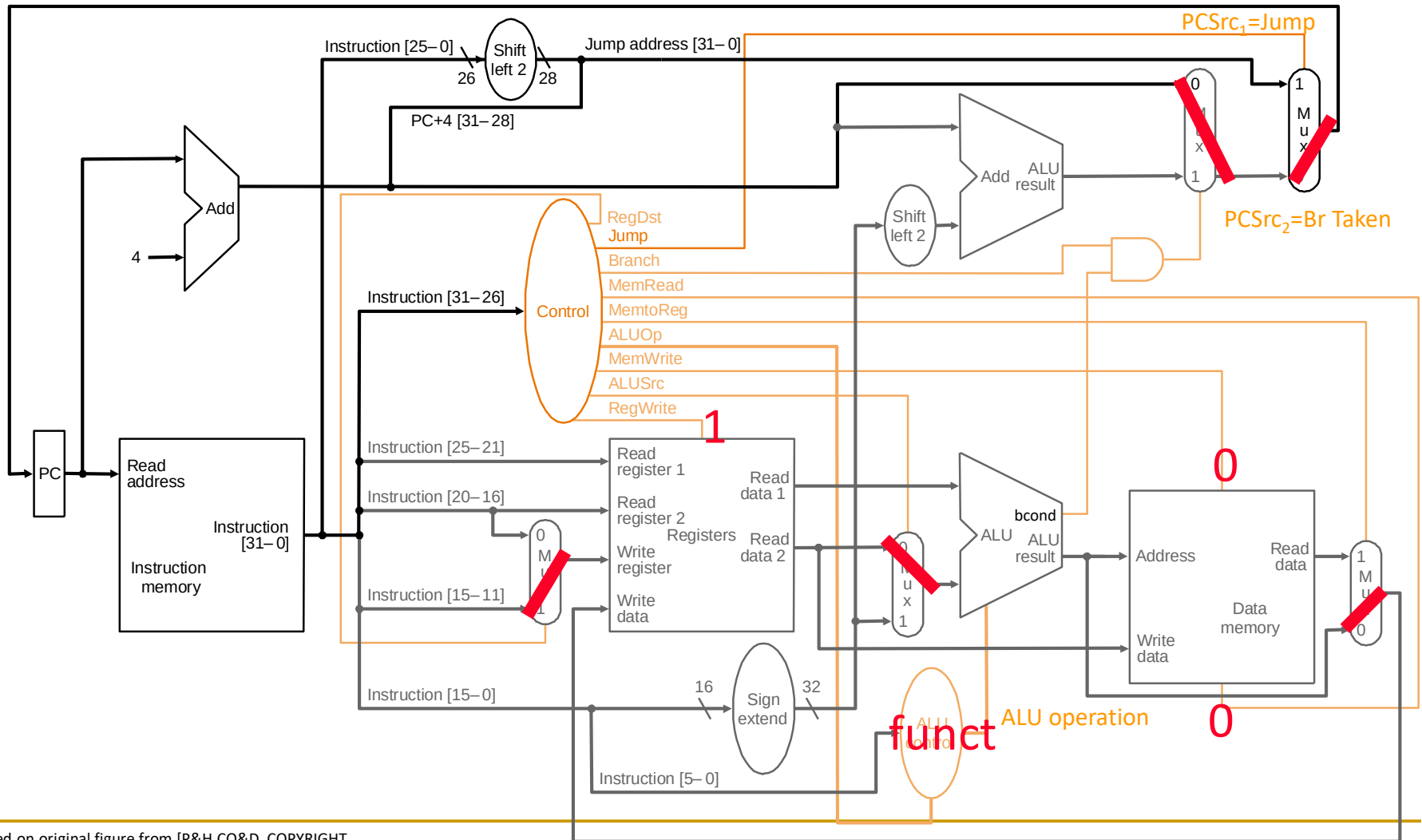
JAL and JALR require additional RegDst and MemtoReg options

# Single-Bit Control Signals (II)

---

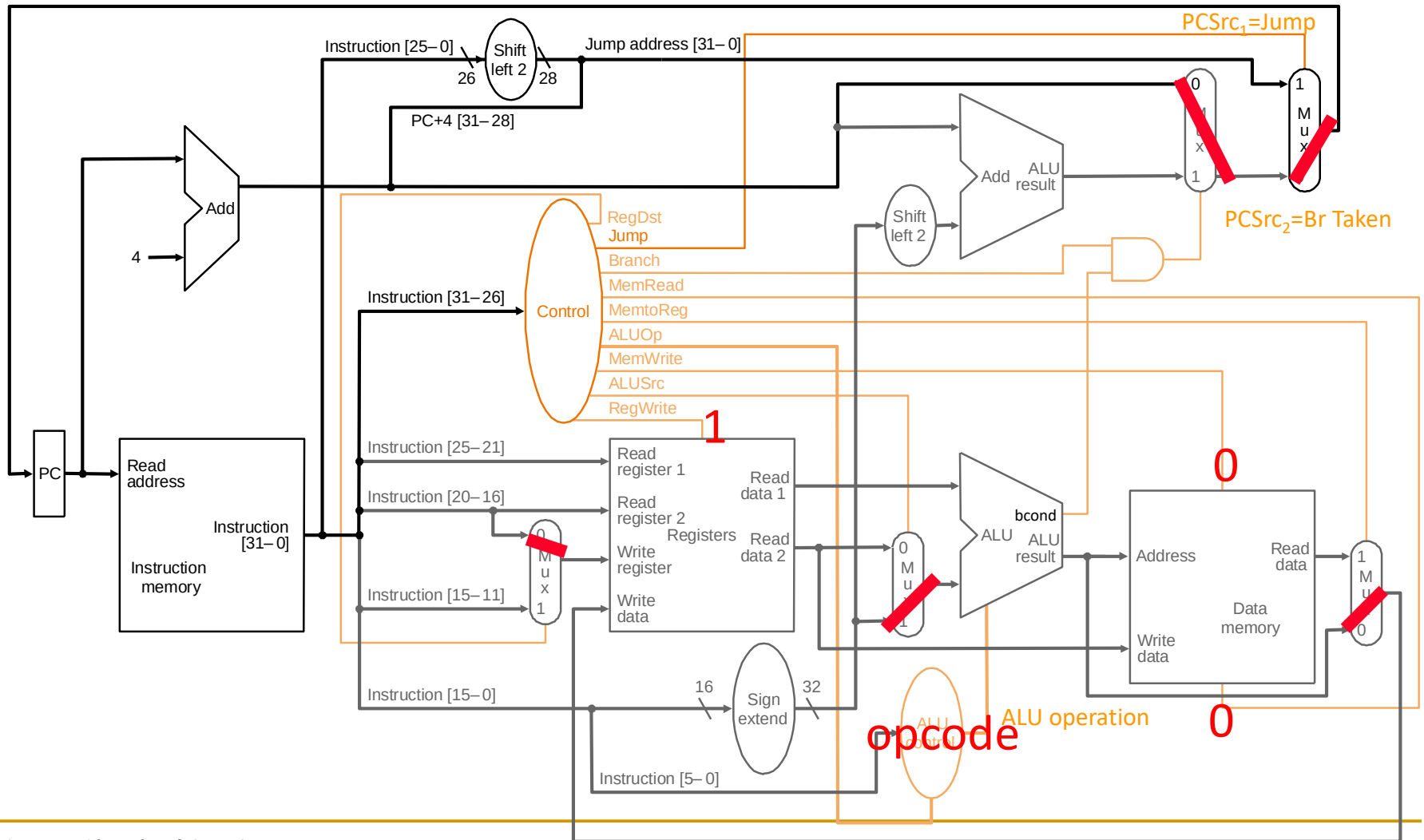
|                    | When De-asserted                      | When asserted                                             | Equation                                                                   |
|--------------------|---------------------------------------|-----------------------------------------------------------|----------------------------------------------------------------------------|
| MemRead            | Memory <b>read disabled</b>           | Memory <b>read port returns load value</b>                | $\text{opcode} == \text{LW}$                                               |
| MemWrite           | Memory <b>write disabled</b>          | Memory <b>write enabled</b>                               | $\text{opcode} == \text{SW}$                                               |
| PCSrc <sub>1</sub> | According to <b>PCSrc<sub>2</sub></b> | next PC is based on <b>26-bit immediate jump target</b>   | $(\text{opcode} == \text{J}) \mid \mid$<br>$(\text{opcode} == \text{JAL})$ |
| PCSrc <sub>2</sub> | next PC = <b>PC + 4</b>               | next PC is based on <b>16-bit immediate branch target</b> | $(\text{opcode} == \text{Bxx}) \ \&\&$<br>“bcond is satisfied”             |

# R-Type ALU

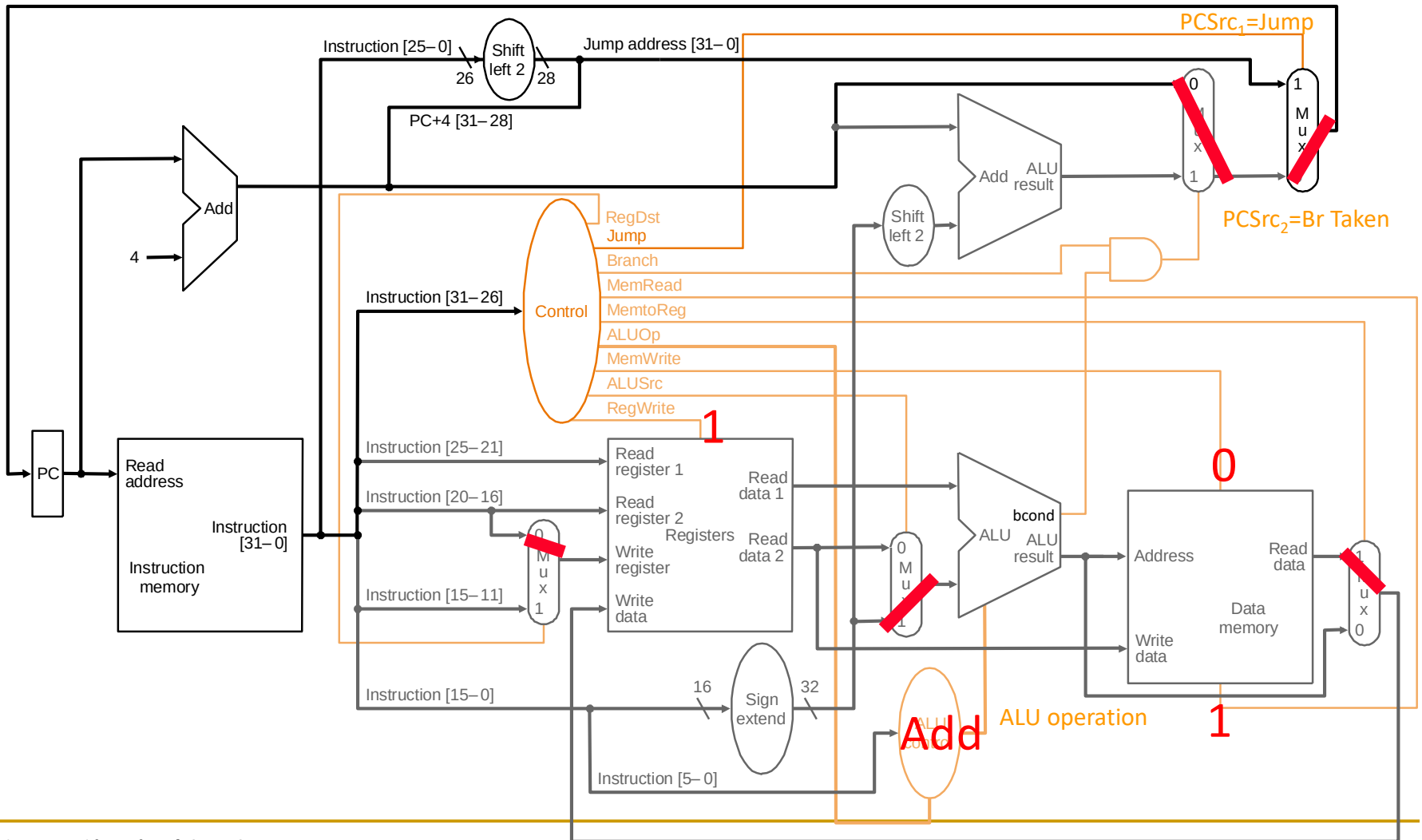


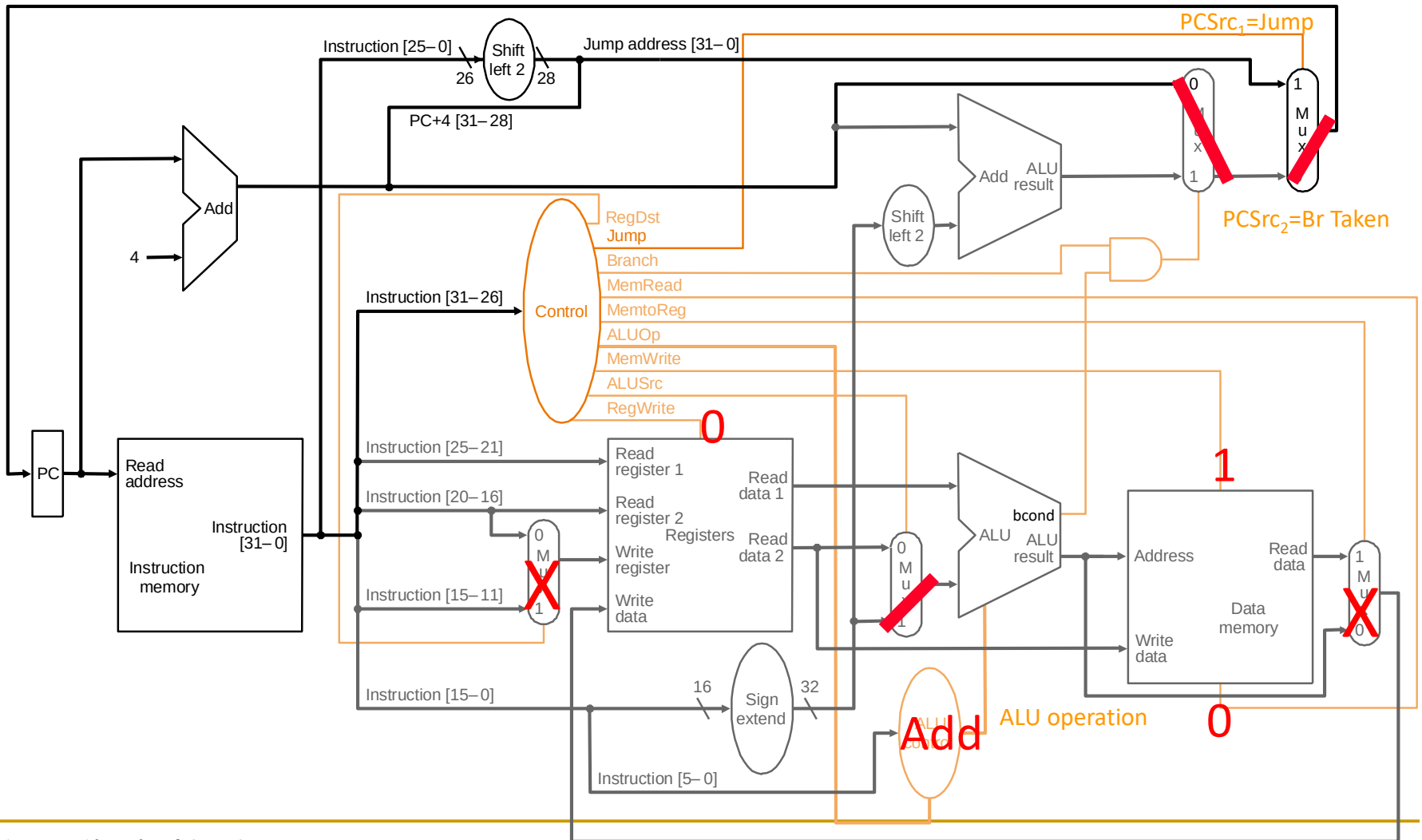
\*\*Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

# I-Type ALU



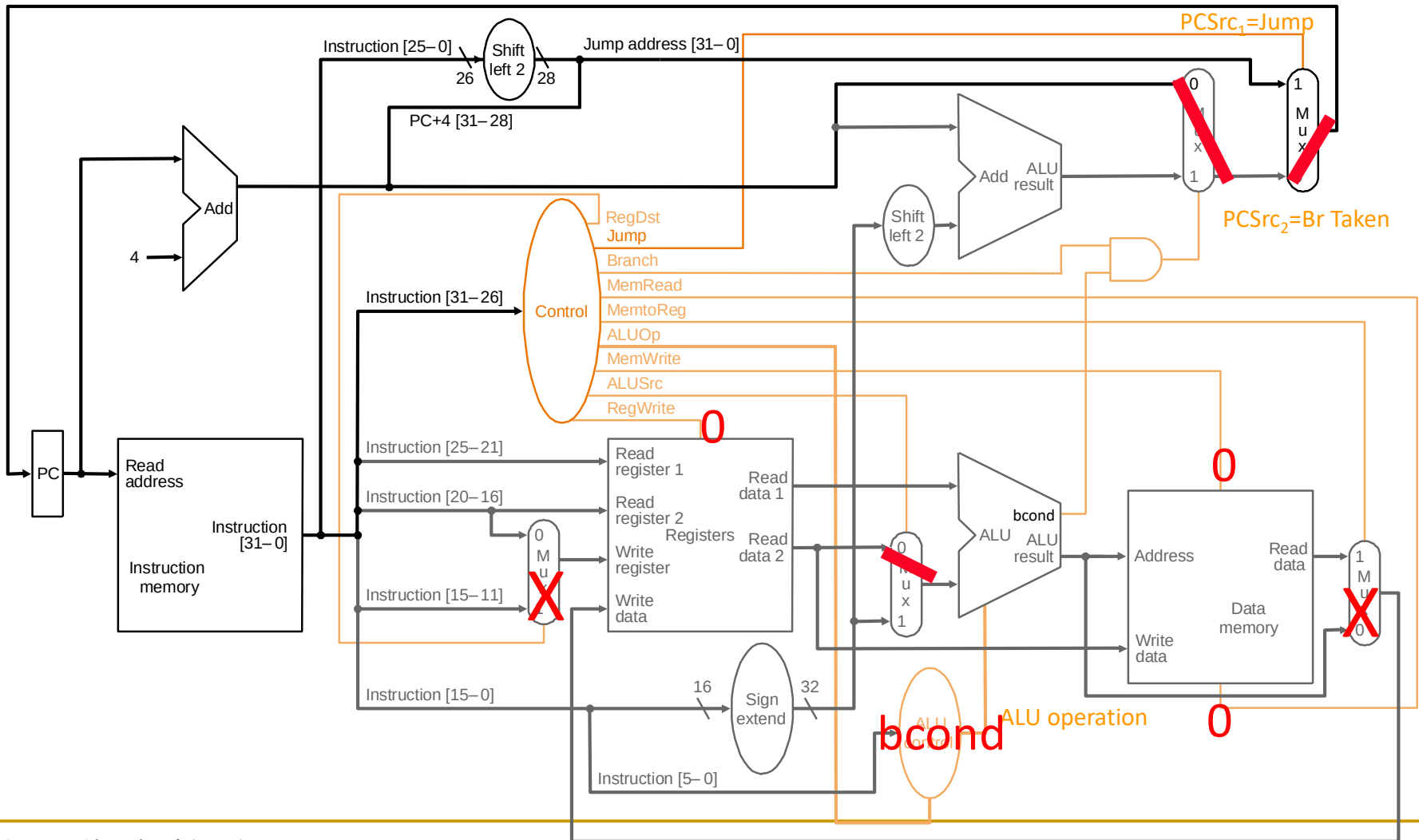
\*\*Based on original figure from [P&H CO&D, COPYRIGHT 2004  
Elsevier. ALL RIGHTS RESERVED.]





# Branch (Not Taken)

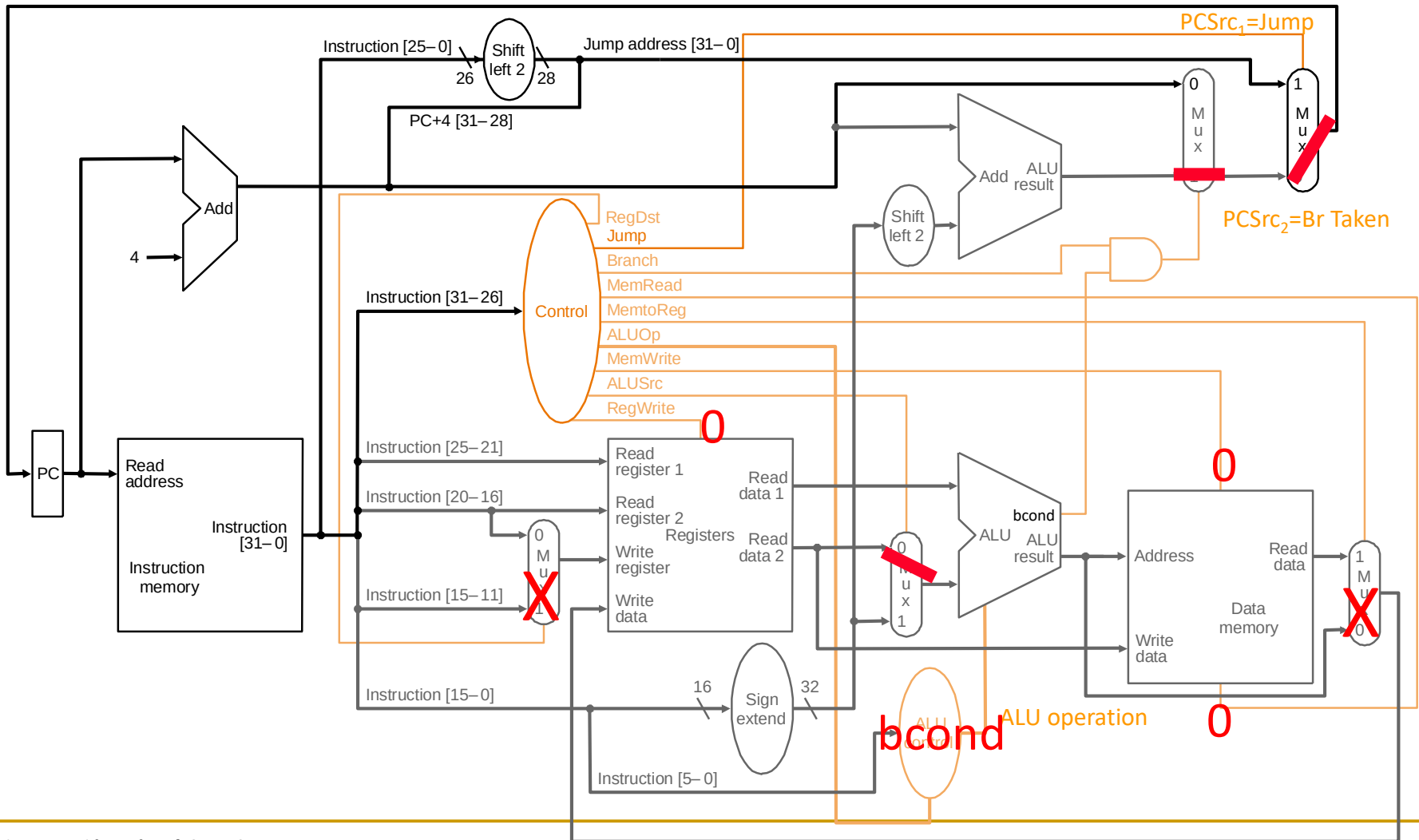
Some control signals are dependent on the processing of data



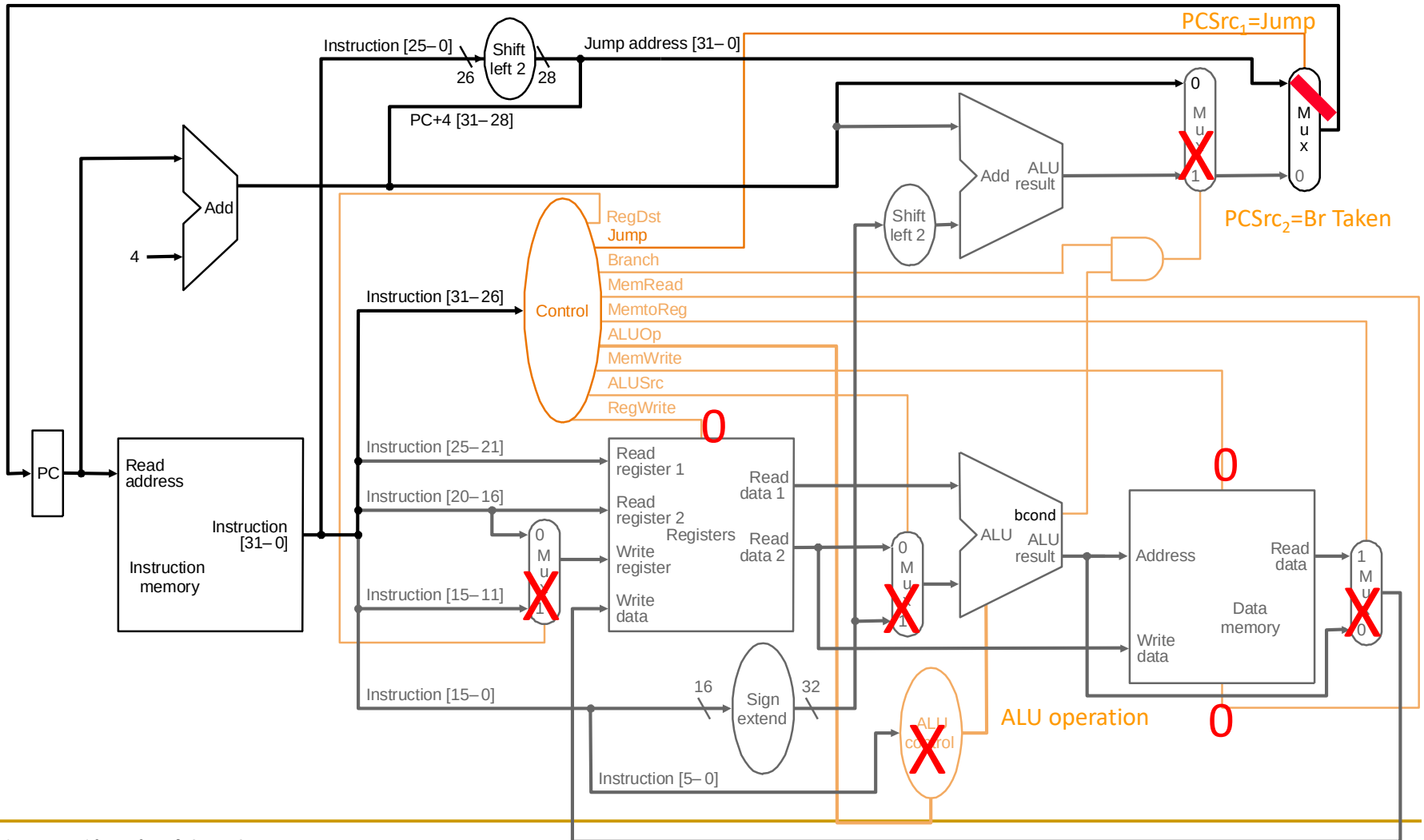
\*\*Based on original figure from [P&H CO&D, COPYRIGHT 2004  
Elsevier. ALL RIGHTS RESERVED.]

# Branch (Taken)

Some control signals are dependent on the processing of data



# Jump



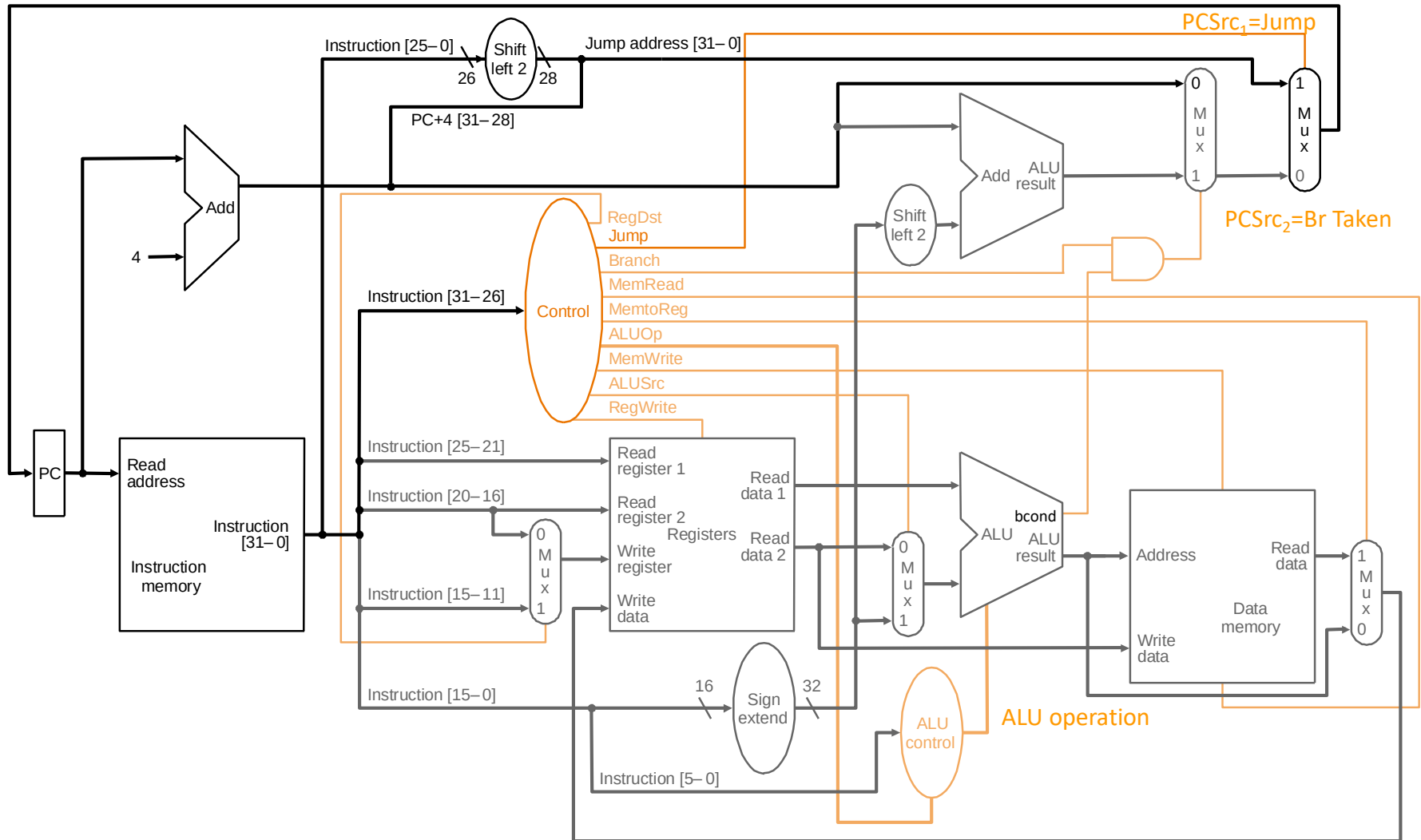
\*\*Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

# What is in That Control Box?

---

- Combinational Logic → **Hardwired Control**
  - Idea: Control signals generated combinatorially based on bits in instruction encoding
- Sequential Logic → **Sequential Control**
  - Idea: A memory structure contains the control signals associated with an instruction
    - Called **Control Store**
- **Both types of control structure can be used in single-cycle processors**
  - Choice depends on latency of each structure + how much on the critical path control signal generation is, etc.

# Review: Complete Single-Cycle Processor



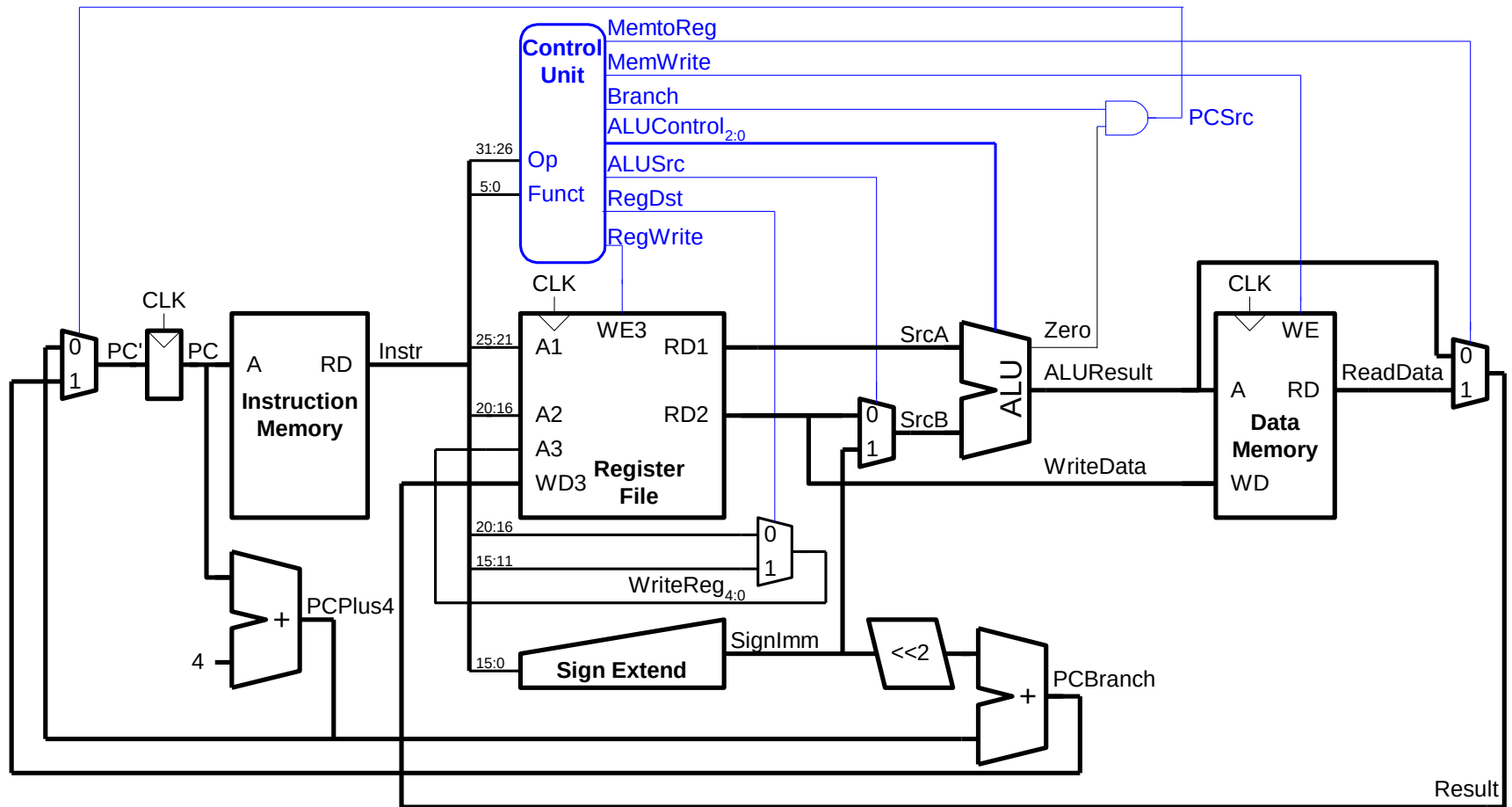
# Another Single-Cycle MIPS Processor (from H&H)

See backup slides to reinforce the concepts we have covered.

They are to complement your reading:

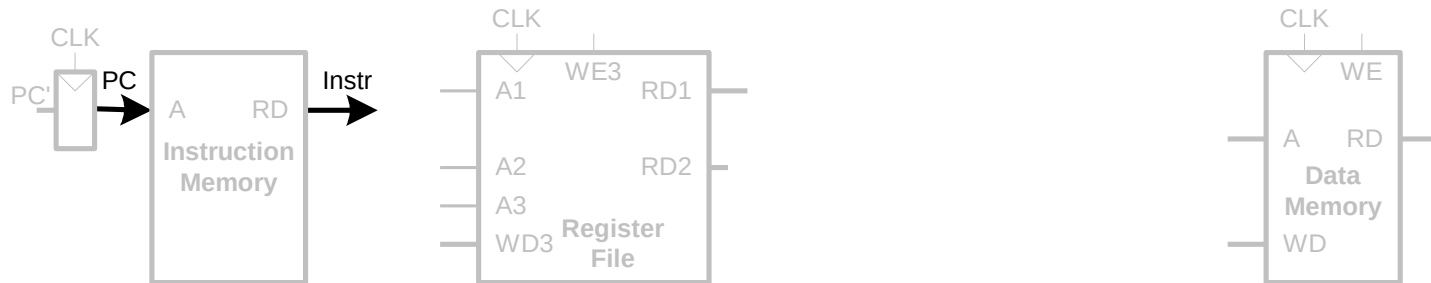
H&H, Chapter 7.1-7.3, 7.6

# Another Complete Single-Cycle Processor



# Example: Single-Cycle Datapath: lw fetch

## ■ **STEP 1:** Fetch instruction



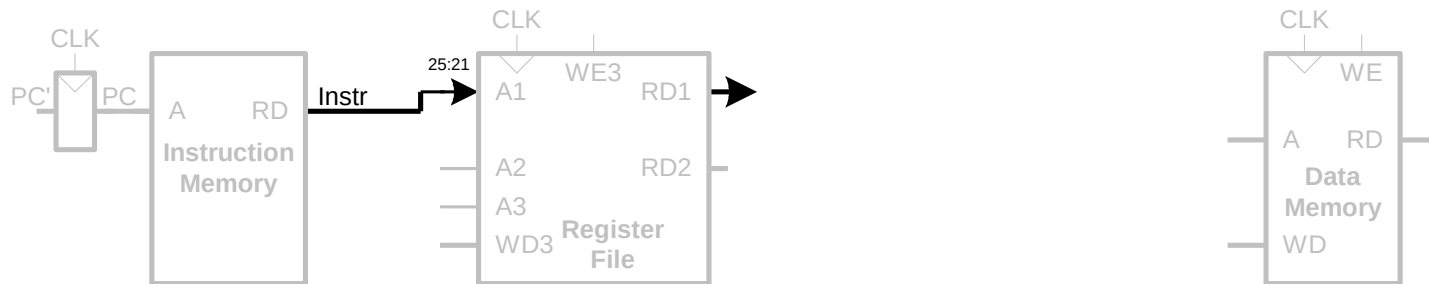
`lw $s3, 1($0)` # read memory word 1 into \$s3

**I-Type**



# Single-Cycle Datapath: lw register read

## ■ **STEP 2:** Read source operands from register file



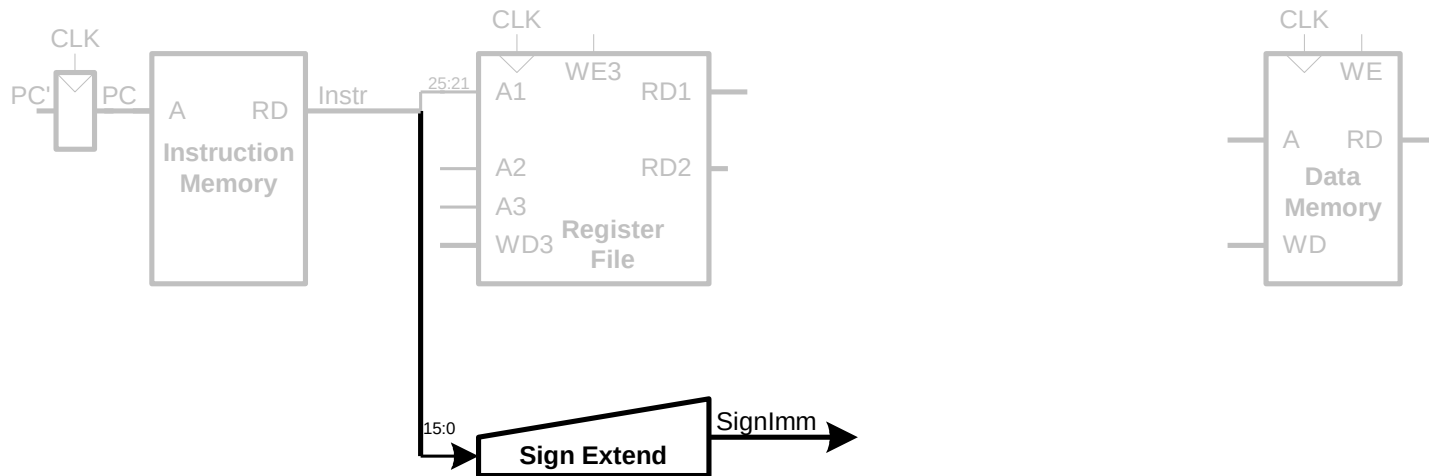
`lw $s3, 1($0)` # read memory word 1 into \$s3

**I-Type**

|        |        |        |         |
|--------|--------|--------|---------|
| op     | rs     | rt     | imm     |
| 6 bits | 5 bits | 5 bits | 16 bits |

# Single-Cycle Datapath: lw immediate

## ■ **STEP 3:** Sign-extend the immediate



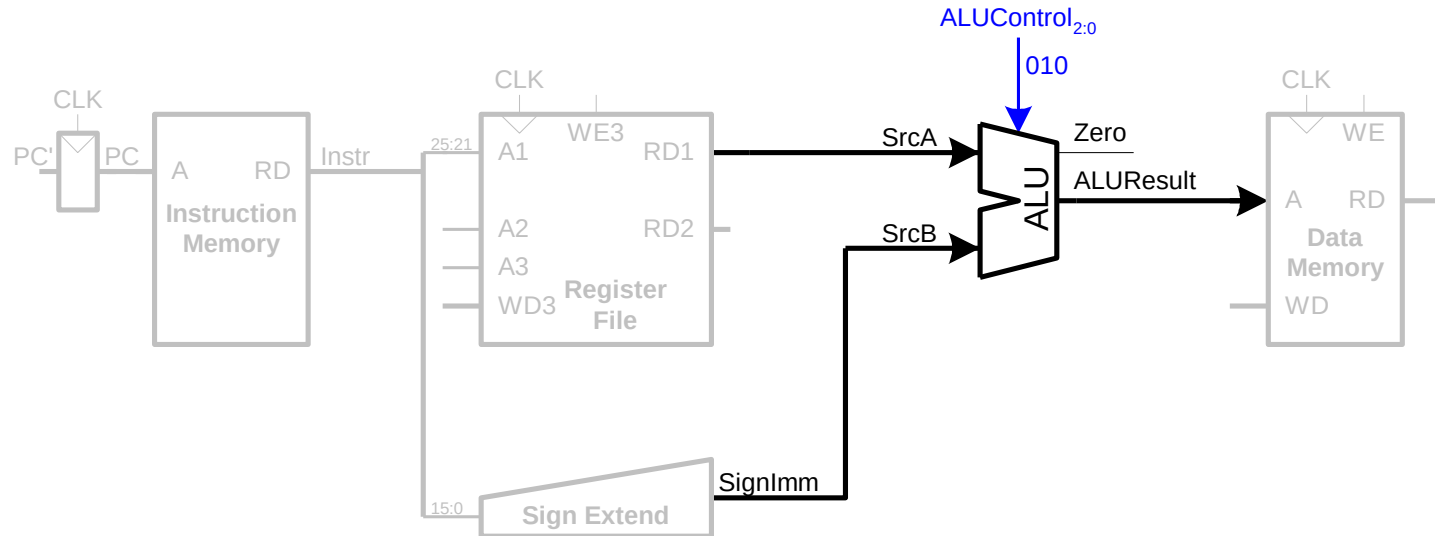
`lw $s3, 1($0)` # read memory word 1 into \$s3

**I-Type**

| op     | rs     | rt     | imm     |
|--------|--------|--------|---------|
| 6 bits | 5 bits | 5 bits | 16 bits |

# Single-Cycle Datapath: lw address

## ■ **STEP 4:** Compute the memory address



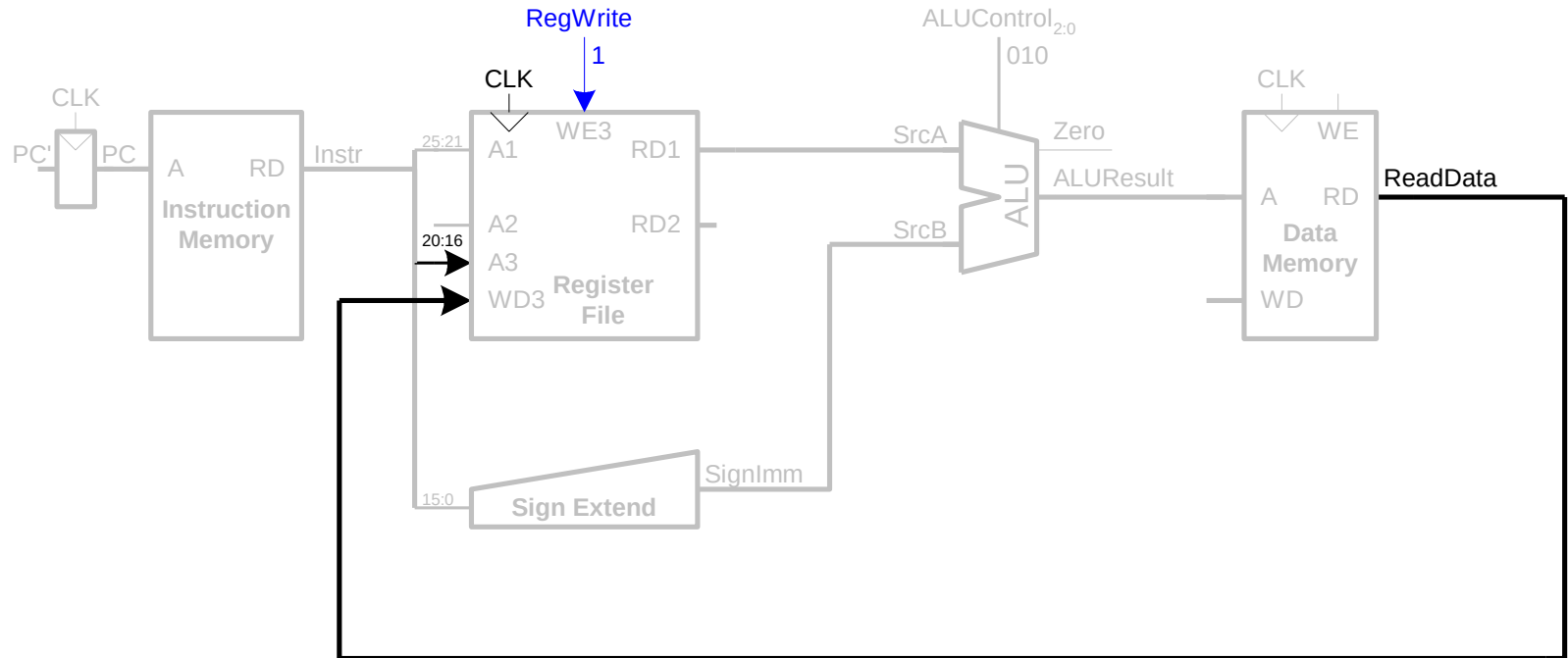
`lw $s3, 1($0)`      # read memory word 1 into \$s3

**I-Type**

| op     | rs     | rt     | imm     |
|--------|--------|--------|---------|
| 6 bits | 5 bits | 5 bits | 16 bits |

# Single-Cycle Datapath: lw memory read

## ■ **STEP 5:** Read from memory and write back to register file



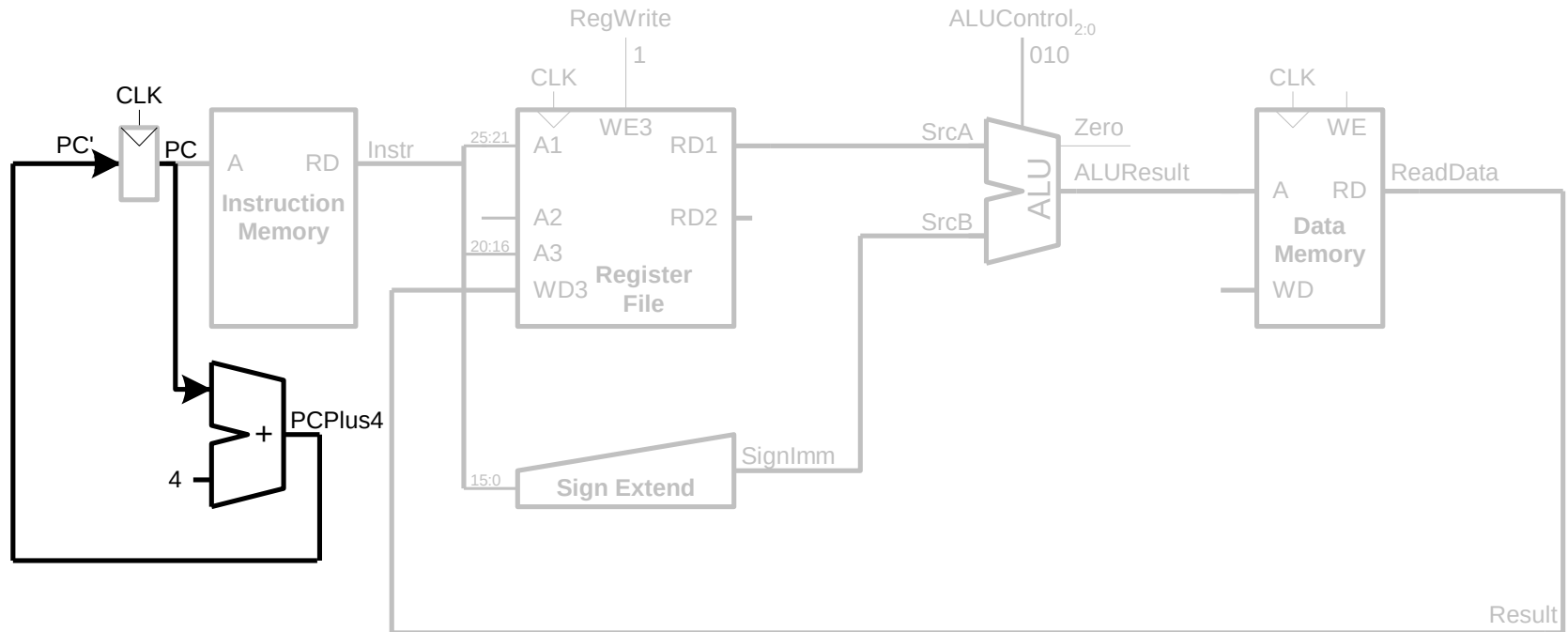
`lw $s3, 1($0)` # read memory word 1 into \$s3

**I-Type**

| op     | rs     | rt     | imm     |
|--------|--------|--------|---------|
| 6 bits | 5 bits | 5 bits | 16 bits |

# Single-Cycle Datapath: lw PC increment

## ■ **STEP 6:** Determine address of next instruction



`lw $s3, 1($0)`      # read memory word 1 into \$s3

**I-Type**

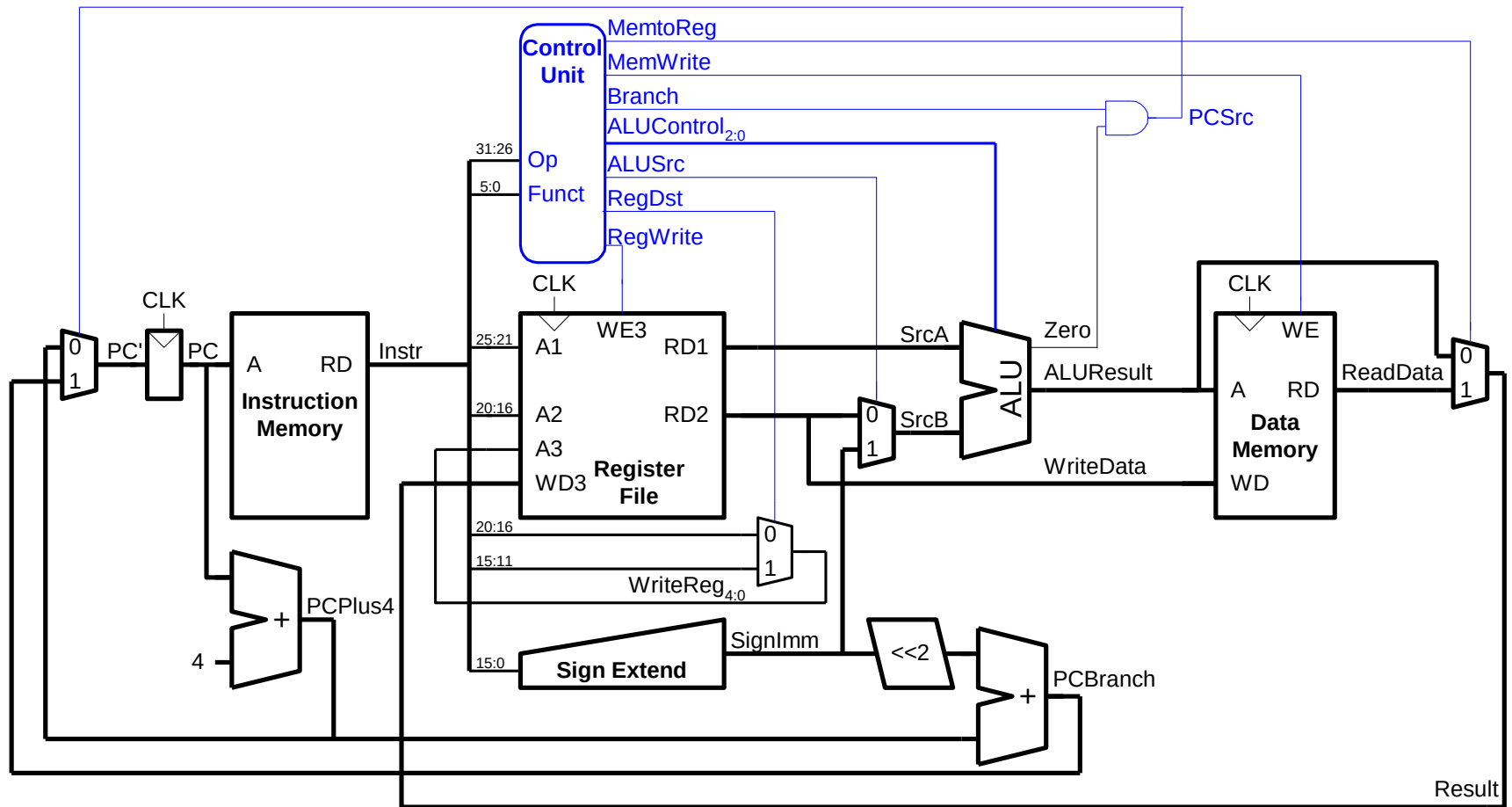
| op     | rs     | rt     | imm     |
|--------|--------|--------|---------|
| 6 bits | 5 bits | 5 bits | 16 bits |

# Similarly, We Need to Design the Control Unit

- **Control signals** are generated by the decoder in control unit

| Instruction | Op <sub>5:0</sub> | RegWrite | RegDst | AluSrc | Branch | MemWrite | MemtoReg | ALUOp <sub>1:0</sub> | Jump |
|-------------|-------------------|----------|--------|--------|--------|----------|----------|----------------------|------|
| R-type      | 000000            | 1        | 1      | 0      | 0      | 0        | 0        | 10                   | 0    |
| lw          | 100011            | 1        | 0      | 1      | 0      | 0        | 1        | 00                   | 0    |
| sw          | 101011            | 0        | X      | 1      | 0      | 1        | X        | 00                   | 0    |
| beq         | 000100            | 0        | X      | 0      | 1      | 0        | X        | 01                   | 0    |
| addi        | 001000            | 1        | 0      | 1      | 0      | 0        | 0        | 00                   | 0    |
| j           | 000010            | 0        | X      | X      | X      | 0        | X        | XX                   | 1    |

# Another Complete Single-Cycle Processor (H&H)

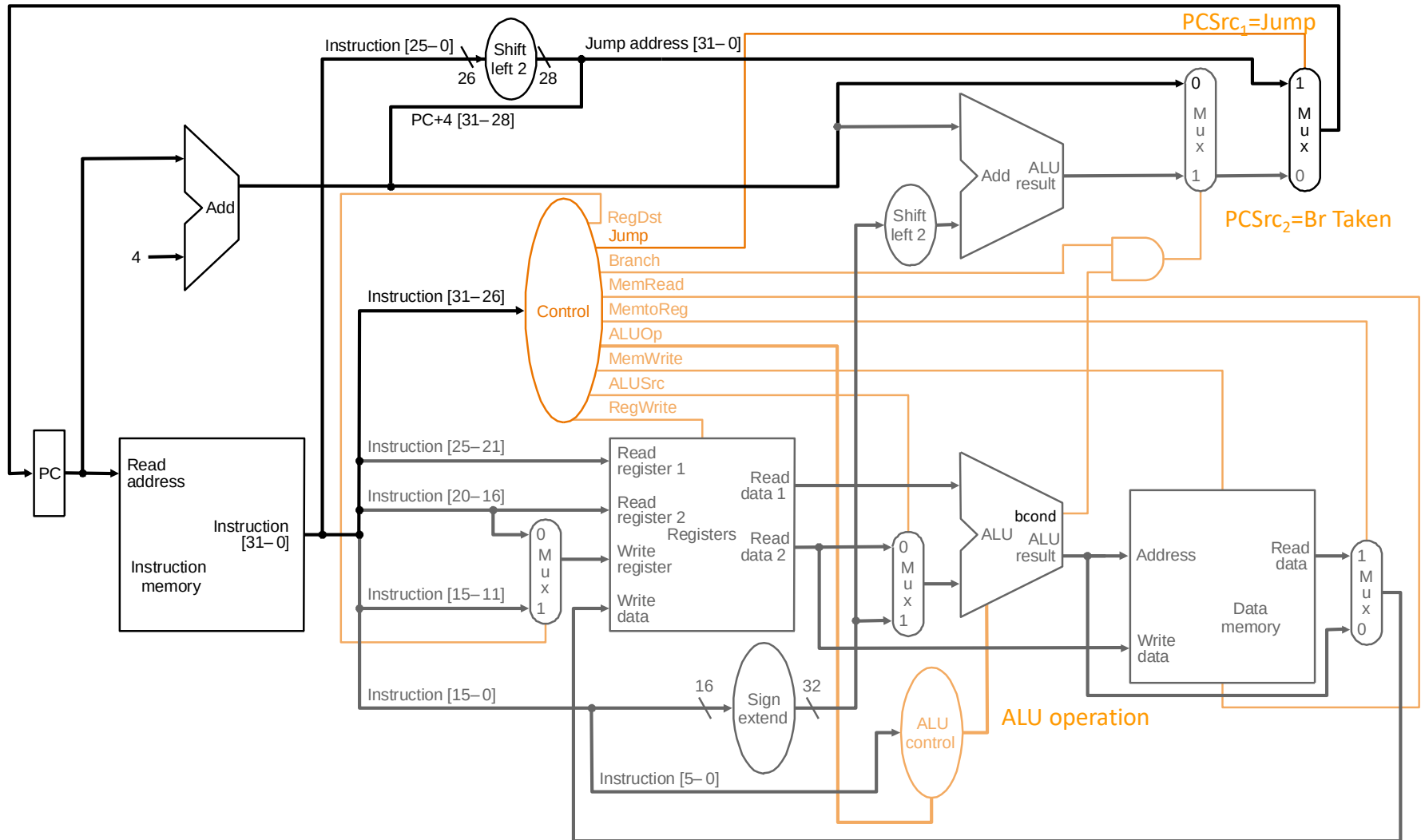


# Your Reading Assignment

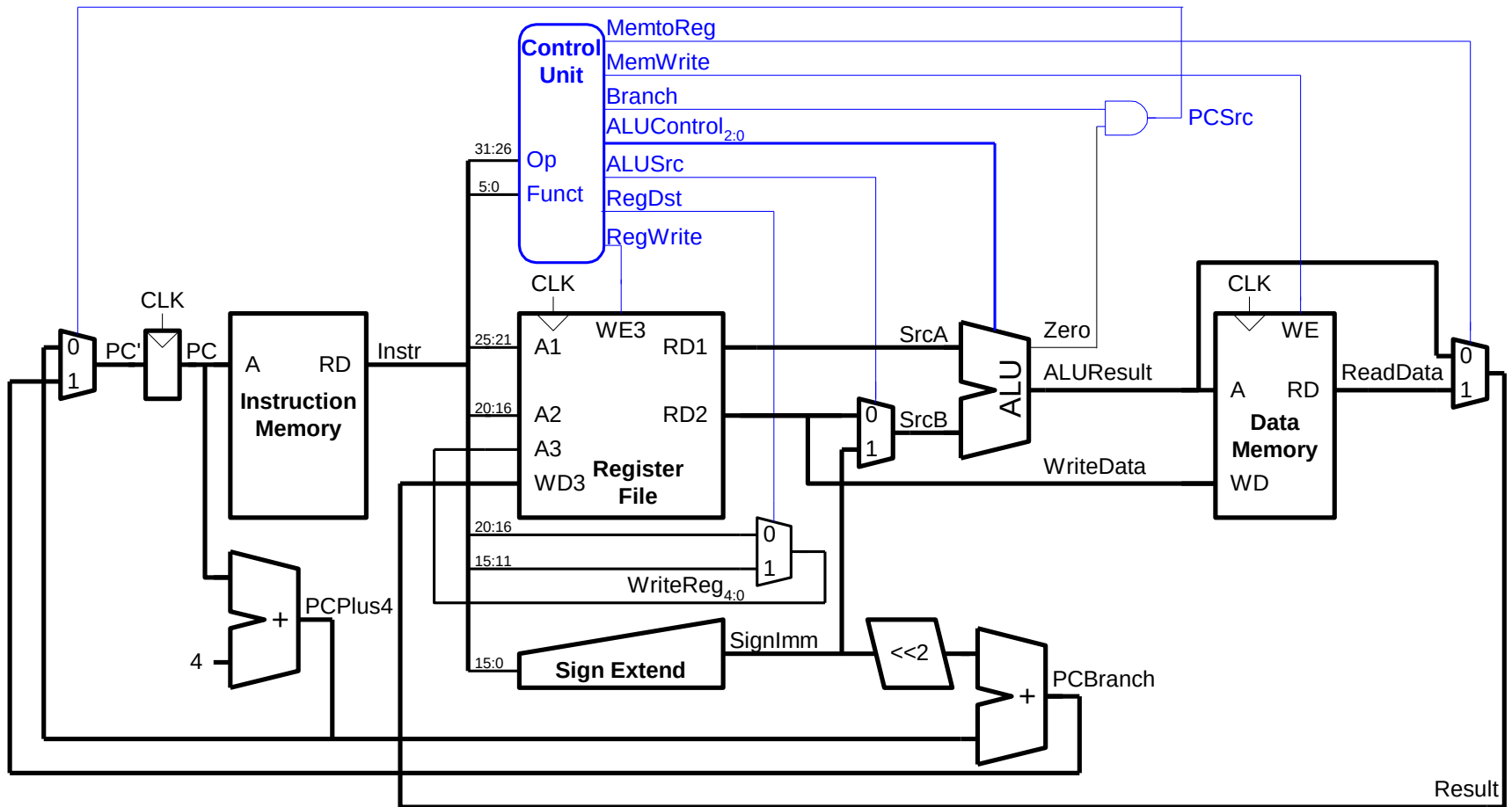
---

- Please go over and internalize the **Lecture Slides & Backup Slides**
- Please do your readings from the H&H Book
  - H&H, Chapter 7.1-7.3, 7.6

# Single-Cycle Uarch I (We Developed in Lectures)



# Single-Cycle Uarch II (In Your Readings)



# Evaluating the Single-Cycle Microarchitecture

# A Single-Cycle Microarchitecture

---

- Is *this* a good idea/design?
- When is this a good design?
- When is this a bad design?
- How can we design a better microarchitecture?

# Performance Analysis Basics

# Recall: Performance Analysis Basics

---

- Execution time of a single instruction
  - **{CPI} x {clock cycle time}**
    - CPI: Number of cycles it takes to execute an instruction
- Execution time of an entire program
  - Sum over all instructions [**{CPI} x {clock cycle time}**]
  - **{# of instructions} x {Average CPI} x {clock cycle time}**

# Processor Performance

- **How fast is my program?**
  - Every program consists of a series of instructions
  - Each instruction needs to be executed

# Processor Performance

## ■ How fast is my program?

- Every program consists of a series of instructions
- Each instruction needs to be executed

## ■ How fast are my instructions?

- Instructions are realized on the hardware
- Each instruction can take one or more clock cycles to complete
- *Cycles per Instruction = CPI*

# Processor Performance

## ■ How fast is my program?

- Every program consists of a series of instructions
- Each instruction needs to be executed

## ■ How fast are my instructions?

- Instructions are realized on the hardware
- Each instruction can take one or more clock cycles to complete
- *Cycles per Instruction = CPI*

## ■ How long is one clock cycle?

- The critical path determines how much time one cycle requires = *clock period*
- $1/\text{clock period} = \text{clock frequency}$  = how many clock cycles are in each second

# Processor Performance

## ■ As a general formula

- Our program consists of executing **N** instructions
- Our processor needs **CPI** cycles (on average) for each instruction
- The clock frequency of the processor is **f**
  - ➔ the clock period is therefore  **$T=1/f$**

# Processor Performance

## ■ As a general formula

- Our program consists of executing **N** instructions
- Our processor needs **CPI** cycles (on average) for each instruction
- The clock frequency of the processor is **f**
  - the clock period is therefore **T=1/f**

## ■ Our program executes in

$$N \times CPI \times (1/f) =$$

$$N \times CPI \times T \text{ seconds}$$

# Performance Analysis of Our Single-Cycle Design

# A Single-Cycle Microarchitecture: Analysis

---

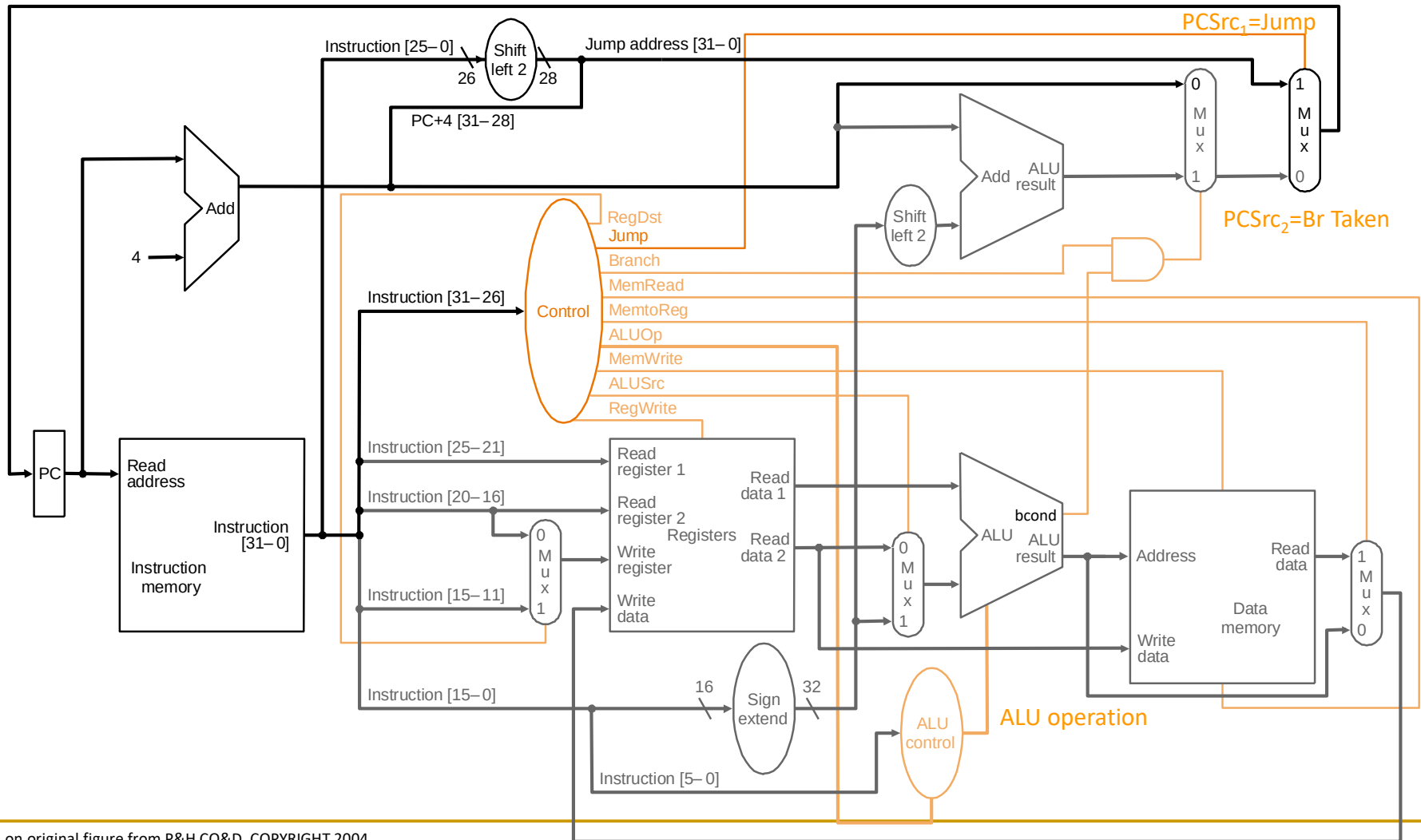
- Every instruction takes 1 cycle to execute
  - CPI (Cycles per instruction) is strictly 1
- How long each instruction takes is determined by how long the slowest instruction takes to execute
  - Even though many instructions do not need that long to execute
- Clock cycle time of the microarchitecture is determined by how long it takes to complete the slowest instruction
  - Critical path of the design is determined by the processing time of the slowest instruction

# What is the Slowest Instruction to Process?

---

- Let's go back to the basics
- All six phases of the instruction processing cycle take a *single machine clock cycle* to complete
  - ❑ Fetch
    - 1. Instruction fetch (IF)
  - ❑ Decode
    - 2. Instruction decode and
  - ❑ Evaluate Address
    - register operand fetch (ID/RF)
  - ❑ Fetch Operands
    - 3. Execute/Evaluate memory address (EX/AG)
  - ❑ Execute
    - 4. Memory operand fetch (MEM)
  - ❑ Store Result
    - 5. Store/writeback result (WB)
- Does every instruction take the same time (latency) to complete?

# Let's Find the Critical Path



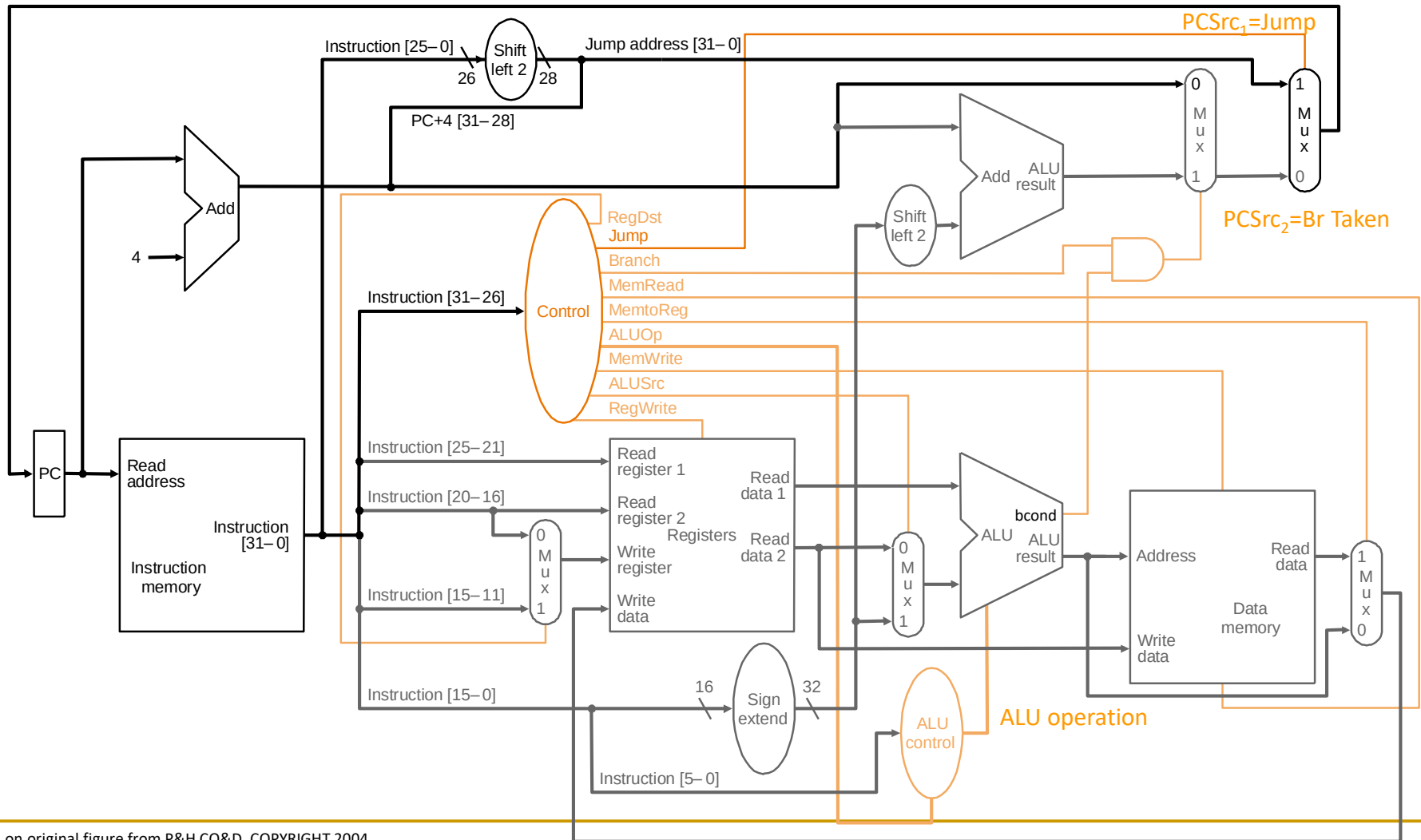
# Example Single-Cycle Datapath Analysis

---

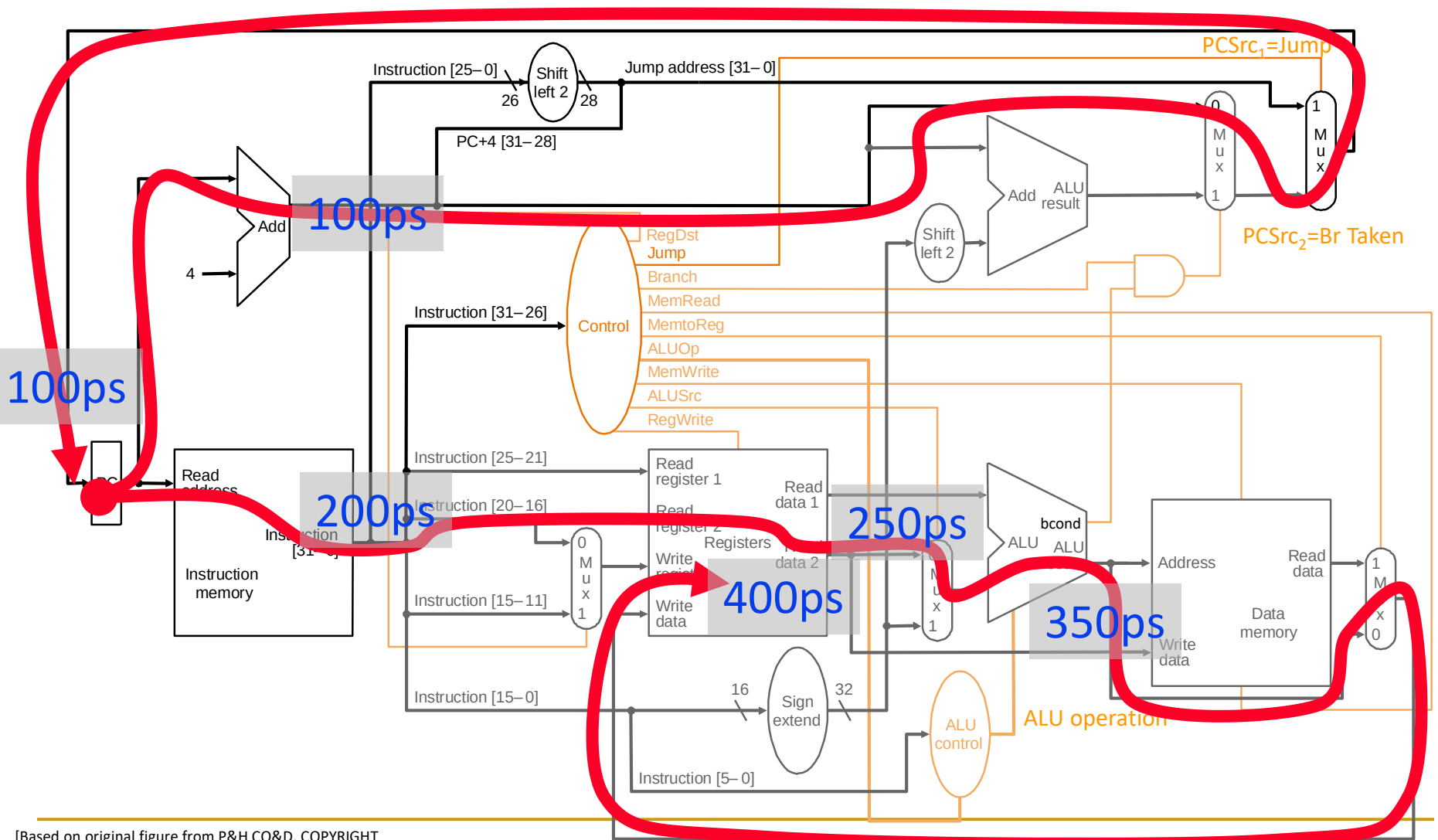
- Assume (for the design in the previous slide)
  - ❑ memory units (read or write): 200 ps
  - ❑ ALU and adders: 100 ps
  - ❑ register file (read or write): 50 ps
  - ❑ other logic or wire delay: 0 ps

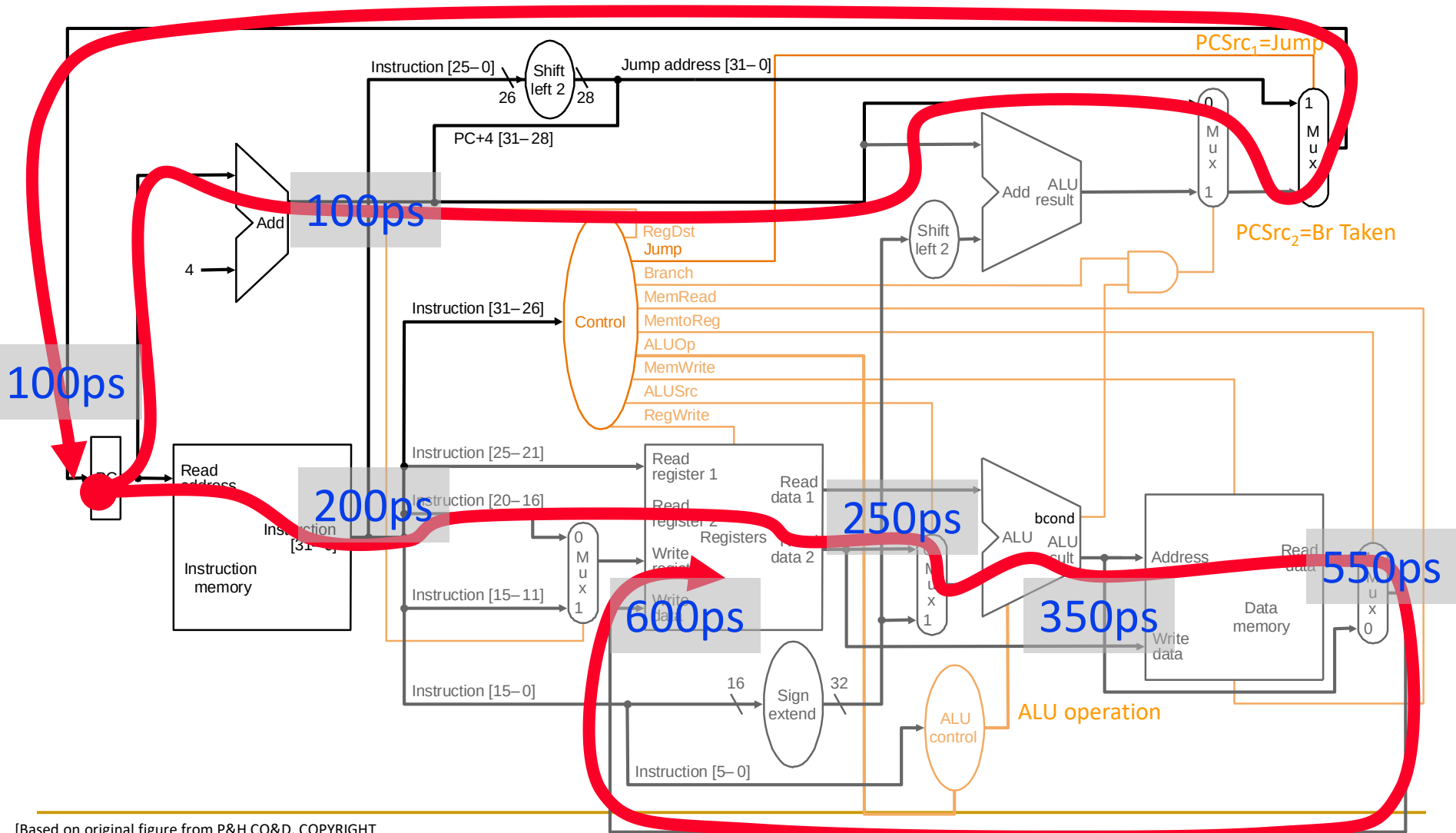
| steps     | IF  | ID | EX  | MEM | WB | Delay |
|-----------|-----|----|-----|-----|----|-------|
| resources | mem | RF | ALU | mem | RF |       |
| R-type    | 200 | 50 | 100 |     | 50 | 400   |
| I-type    | 200 | 50 | 100 |     | 50 | 400   |
| LW        | 200 | 50 | 100 | 200 | 50 | 600   |
| SW        | 200 | 50 | 100 | 200 |    | 550   |
| Branch    | 200 | 50 | 100 |     |    | 350   |
| Jump      | 200 |    |     |     |    | 200   |

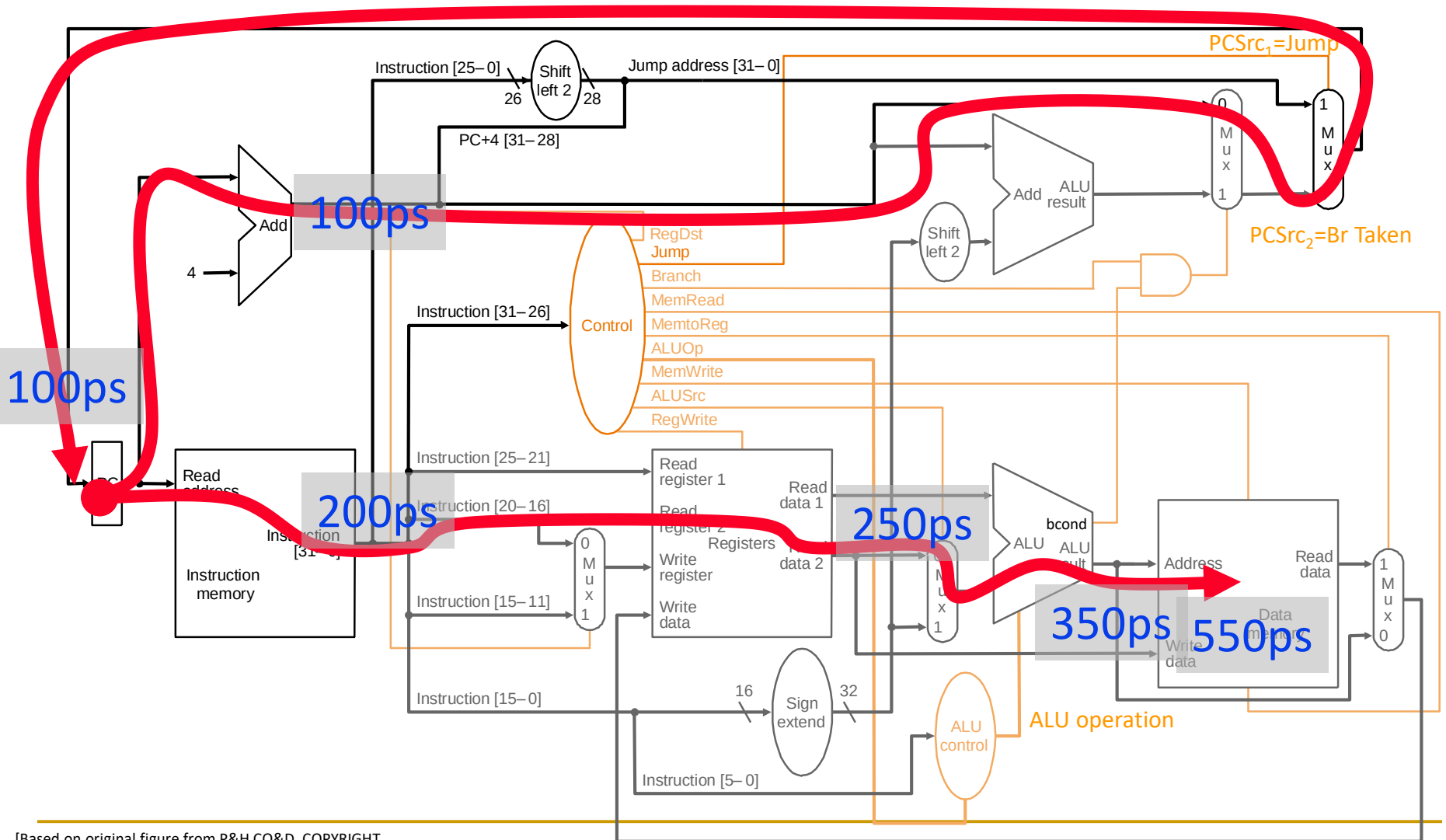
# Let's Find the Critical Path



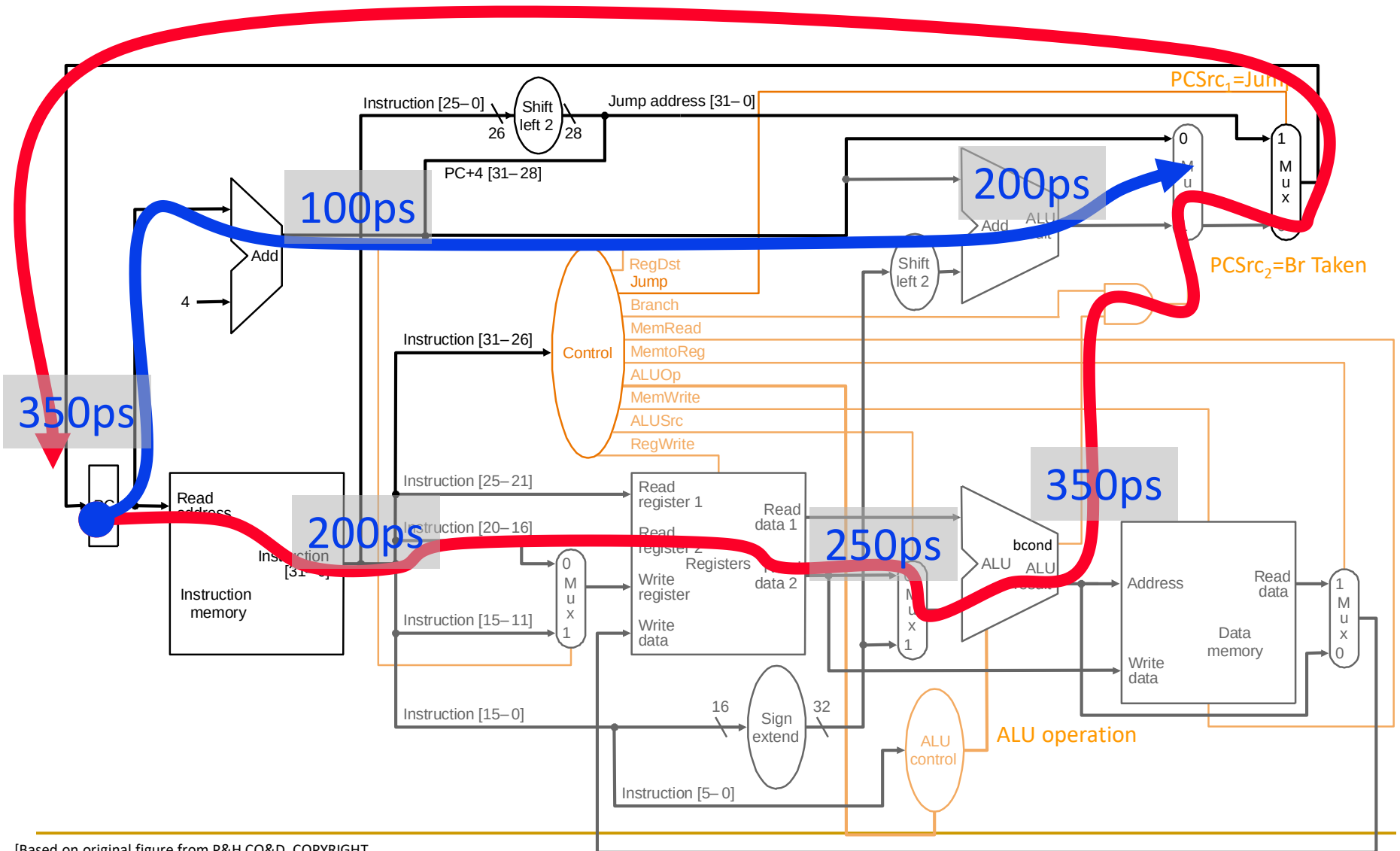
# R-Type and I-Type ALU



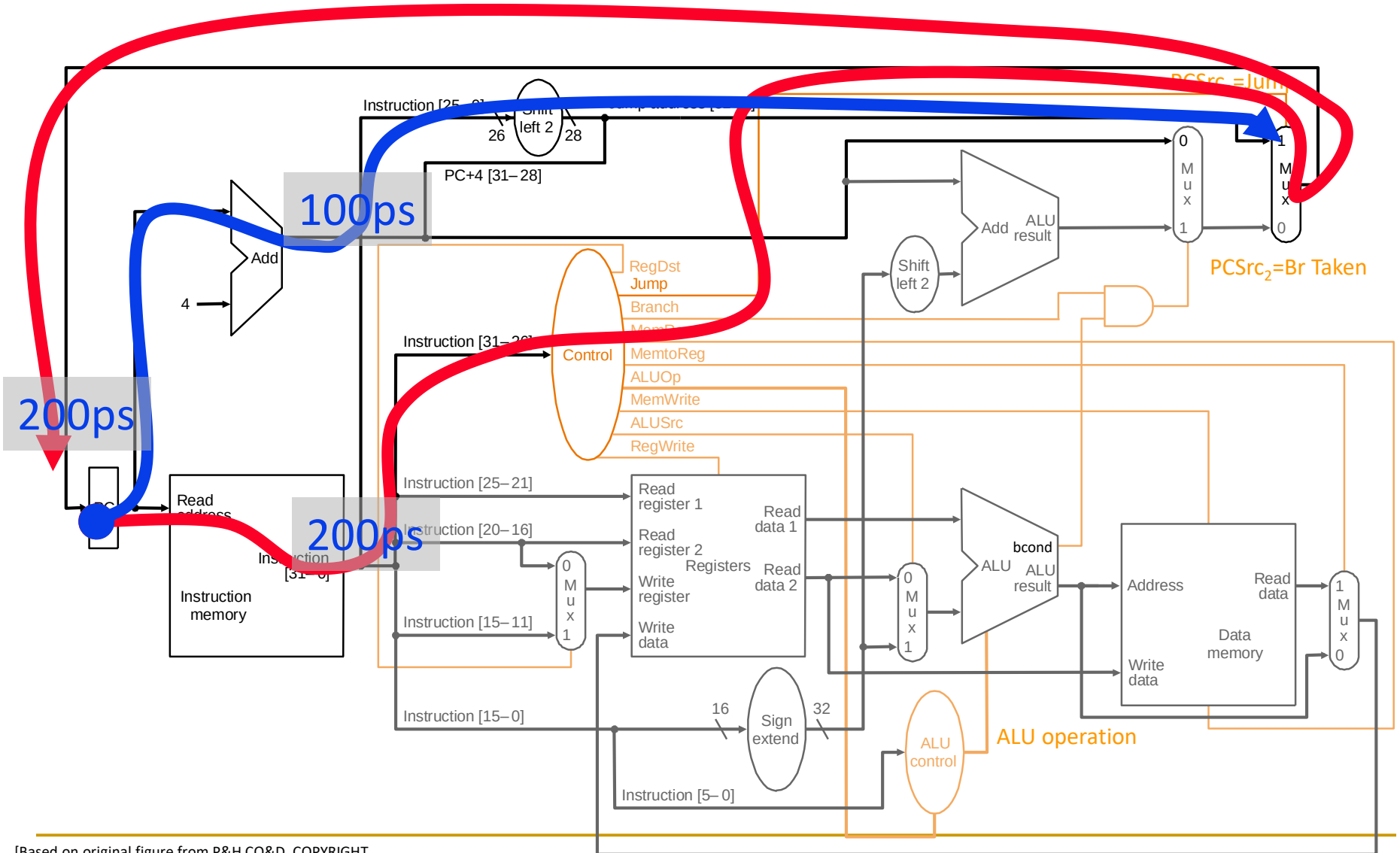




# Branch Taken



# Jump



# Example Single-Cycle Datapath Analysis

---

- Assume (for the design in the previous slide)
  - ❑ memory units (read or write): 200 ps
  - ❑ ALU and adders: 100 ps
  - ❑ register file (read or write): 50 ps
  - ❑ other logic or wire delay: 0 ps

| steps     | IF  | ID | EX  | MEM | WB | Delay |
|-----------|-----|----|-----|-----|----|-------|
| resources | mem | RF | ALU | mem | RF |       |
| R-type    | 200 | 50 | 100 |     | 50 | 400   |
| I-type    | 200 | 50 | 100 |     | 50 | 400   |
| LW        | 200 | 50 | 100 | 200 | 50 | 600   |
| SW        | 200 | 50 | 100 | 200 |    | 550   |
| Branch    | 200 | 50 | 100 |     |    | 350   |
| Jump      | 200 |    |     |     |    | 200   |

# What About Control Logic?

---

- How does that affect the critical path?
- Food for thought for you:
  - Can control logic be on the critical path?
  - Historical example:
    - CDC 5600: control store access took too long...

**We Covered Until Here in Lecture**

# Digital Design & Computer Arch.

## Lecture 10: Microarchitecture Fundamentals & Design

Dr. Mohammad Sadrosadati

Prof. Onur Mutlu

ETH Zürich

Spring 2026

20 March 2026

**Further Slides for Your Own Study  
(May Be Covered in Future Lectures)**

# What is Really the Slowest Instruction to Process?

---

- Real world: **Memory is slow (not magic)**
- What if memory *sometimes* takes 150ns to access?
- Does it make sense to have a simple register to register add or jump to take {150ns + all else to perform a memory operation}?
- And, what if you need to access memory more than once to process an instruction?
  - Which instructions require this?

# Single Cycle uArch: Complexity

---

## ■ Contrived

- All instructions run as slow as the slowest instruction

## ■ Inefficient

- All instructions run as slow as the slowest instruction
- Must provide worst-case combinational resources in parallel as required by any instruction
- Need to replicate a resource if it is needed more than once by an instruction during different parts of the instruction processing cycle

## ■ Not necessarily the simplest way to implement an ISA

- Tough for complex instructions, e.g., REP MOVSB (x86) or INDEX (VAX)

## ■ Not easy to optimize/improve performance

- Optimizing the common case (frequent instructions) does not work
- Need to optimize the worst case all the time

# (Micro)architecture Design Principles

---

## ■ Critical path design

- Find and **decrease the maximum combinational logic delay**
- Break a path into multiple cycles if it takes too long

## ■ Bread and butter (common case) design

- **Spend time and resources on where it matters most**
  - i.e., improve what the machine is really designed to do
- Common case vs. uncommon case

## ■ Balanced design

- **Balance** instruction/data flow through hardware components
- **Design to eliminate bottlenecks:** balance the hardware for the work

# Single-Cycle Design vs. Design Principles

---

- Critical path design
- Bread and butter (common case) design
- Balanced design

*How does a single-cycle microarchitecture fare with respect to these principles?*

# Aside: System Design Principles

---

- When designing computer systems/architectures, it is important to follow good principles
  - Actually, this is true for **any system design**
    - Real architectures, buildings, bridges, train stations, ...
    - Good consumer products
    - Security & safety-critical systems
    - Decision making systems
    - ...
- Remember: “principled design” from our second lecture
  - Frank Lloyd Wright: “architecture [...] based upon **principle**, and not upon **precedent**”

# Aside: Principled Architecture

---

- “architecture [...] based upon **principle**, and not upon **precedent**”



# This

---



# That

---



# Recall: Takeaways

---

- It all starts from the basic building blocks and design principles
- And, knowledge of how to use, apply, enhance them
- Underlying technology might change (e.g., steel vs. wood)
  - but methods of taking advantage of technology bear resemblance
  - methods used for design depend on the principles employed

# Aside: System Design Principles

---

- We will continue to cover key principles in this course
- Here are some references where you can learn more
- Yale Patt, "Requirements, Bottlenecks, and Good Fortune: Agents for Microprocessor Evolution," Proc. of IEEE, 2001. (Levels of transformation, design point, etc)
- Mike Flynn, "Very High-Speed Computing Systems," Proc. of IEEE, 1966. (Flynn's Bottleneck → Balanced design)
- Gene M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," AFIPS Conference, April 1967. (Amdahl's Law → Common-case design)
- Butler W. Lampson, "Hints for Computer System Design," ACM Operating Systems Review, 1983.

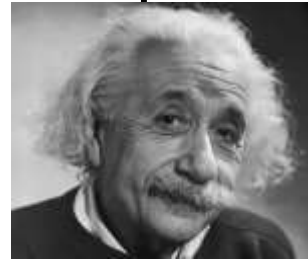
# A Key System Design Principle

---

- **Keep it simple**

- “Everything should be made as simple as possible, **but no simpler.**”

- Albert Einstein (paraphrased)



- And, **keep it low cost**: “An engineer is a person who can do for a dime what any fool can do for a dollar.”

- For more, see:

- Butler W. Lampson, “**Hints for Computer System Design**,” ACM Operating Systems Review, 1983, updated 2021.

- <https://arxiv.org/pdf/2011.02455>



Can We Do Better?

# Multi-Cycle Microarchitectures

# Multi-Cycle Microarchitectures

---

- Goal: Let each instruction take (close to) only as much time it really needs
- Idea
  - Determine clock cycle time independently of instruction processing time
  - Each instruction takes as many clock cycles as it needs to take
    - Multiple state transitions per instruction
    - The states followed by each instruction is different

# Recall: The “Process Instruction” Step

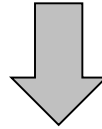
---

- ISA specifies abstractly what  $AS'$  should be, given an instruction and  $AS$ 
  - It defines an **abstract finite state machine** where
    - State = programmer-visible state
    - Next-state logic = instruction execution specification
  - From ISA point of view, there are no “intermediate states” between  $AS$  and  $AS'$  during instruction execution
    - One state transition per instruction
- Microarchitecture implements how  $AS$  is transformed to  $AS'$ 
  - There are many choices in implementation
  - We can have programmer-invisible state to optimize the speed of instruction execution: **multiple** state transitions per instruction
    - Choice 1:  $AS \rightarrow AS'$  (transform  $AS$  to  $AS'$  in a single clock cycle)
    - Choice 2:  $AS \rightarrow AS+MS1 \rightarrow AS+MS2 \rightarrow AS+MS3 \rightarrow AS'$  (take multiple clock cycles to transform  $AS$  to  $AS'$ )

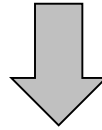
# Multi-Cycle Microarchitecture

---

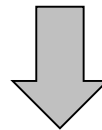
AS = Architectural (programmer visible) state  
at the beginning of an instruction



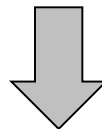
Step 1: Process part of instruction in one clock cycle



Step 2: Process part of instruction in the next clock cycle

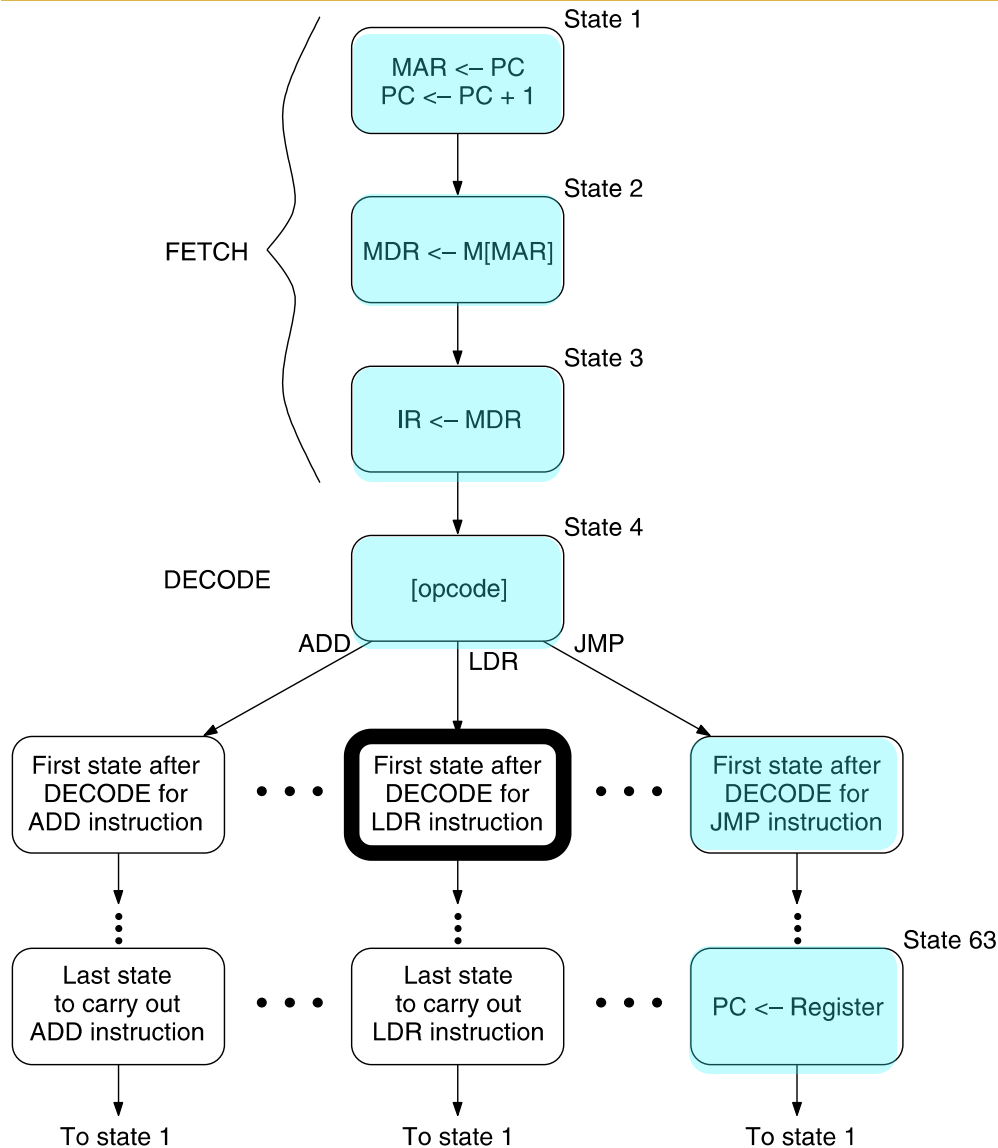


...



AS' = Architectural (programmer visible) state  
at the end of a clock cycle

# Recall: Control of the Instruction Cycle



- State 1
  - The FSM asserts GatePC and LD.MAR
  - It selects input (+1) in PCMUX and asserts LD.PC
- State 2
  - MDR is loaded with the instruction
- State 3
  - The FSM asserts GateMDR and LD.IR
- State 4
  - The FSM goes to next state depending on opcode
- State 63
  - JMP loads register into PC
- Full state diagram in Patt&Pattell, Appendix C

Figure 4.4 An abbreviated state diagram of the LC-3

**This is an FSM Controlling a Multi-Cycle LC-3 Microarchitecture**

# Recall: Full State Machine for LC-3b

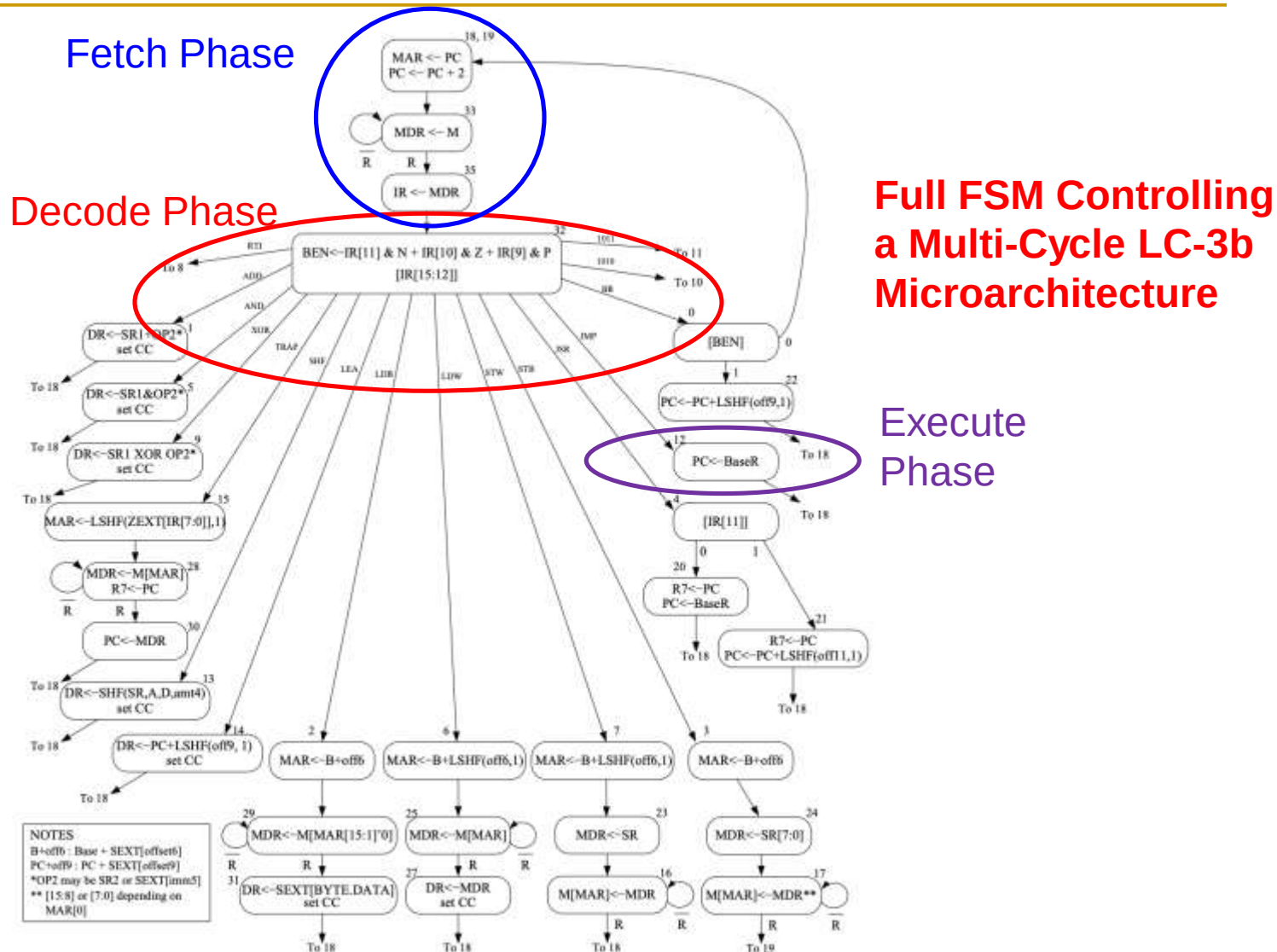


Figure C.2: A state machine for the LC-3b

Diagram illustrating the internal components and data flow of a computer system, showing the interaction between the Processor, Memory, Input, and Output units.

**Processor Components:**

- Control Unit (Red Box):**
  - Program Counter (PC):** Receives **GatePC** and outputs **LD.PC** to the **PCMUX**.
  - PCMUX (Program Counter Multiplexer):** Receives **LD.PC** and **LD.R** from the **IR**. It outputs to the **PC** and the **ALU**.
  - Finite State Machine (FSM):** Receives **CLK** and **LD.R** from the **IR**. It outputs control signals to the **PCMUX**, **REG FILE**, **SR2MUX**, and **ALU**.
  - Instruction Register (IR):** Receives **LD.R** from the **PCMUX** and outputs **LD.R** to the **FSM**.
- Processing Unit (Blue Box):**
  - REG FILE (Register File):** Receives **LD.REG** and **SR2** from the **FSM**. It outputs **SR1** and **SR2** to the **SR2MUX**.
  - SR2MUX (Status Register 2 Multiplexer):** Receives **SR2** from the **REG FILE** and **LD.R** from the **FSM**. It outputs to the **ALU**.
  - ALU (Arithmetic Logic Unit):** Receives **LD.R** from the **FSM** and **SR2MUX**. It outputs **GateALU** to the **PROCESSOR BUS**.
  - ALUK (ALU Control):** Receives **LD.R** from the **FSM** and outputs to the **ALU**.

**Memory and I/O Components:**

- Memory (Green Box):**
  - MDR (Memory Data Register):** Receives **LD.MDR** from the **PROCESSOR BUS** and outputs **GateMDR** to the **PROCESSOR BUS**.
  - MAR (Memory Address Register):** Receives **LD.MAR** from the **PROCESSOR BUS** and outputs **MEM.EN, R.W** to the **Memory**.
- Input (Yellow Box):**
  - KBDR (Keyboard Buffer Data Register):** Receives **LD.KBDR** from the **PROCESSOR BUS** and outputs **GateKBDR** to the **PROCESSOR BUS**.
  - KBSR (Keyboard Buffer Status Register):** Receives **LD.KBSR** from the **PROCESSOR BUS** and outputs **GateKBSR** to the **PROCESSOR BUS**.
- Output (Grey Box):**
  - DDR (Data Register):** Receives **LD.DDR** from the **PROCESSOR BUS** and outputs **GateDDR** to the **PROCESSOR BUS**.
  - DSR (Data Status Register):** Receives **LD.DSR** from the **PROCESSOR BUS** and outputs **GateDSR** to the **PROCESSOR BUS**.

**Interconnections:**

- PROCESSOR BUS:** A 16-bit bus connecting the **Processor**, **Memory**, **Input**, and **Output** units.
- 16-bit addressable:** The **Memory** unit is 16-bit addressable.
- GatePC, GateMDR, GateKBDR, GateKBSR, GateDDR, GateDSR:** Control signals from the **Processor** to the **Memory**, **Input**, and **Output** units.
- LD.PC, LD.REG, LD.R, LD.MDR, LD.MAR, LD.KBDR, LD.KBSR, LD.DDR, LD.DSR:** Data signals from the **Processor** to the **Memory**, **Input**, and **Output** units.

124

# Benefits of Multi-Cycle Design

---

## ■ Critical path design

- ❑ Can keep reducing the critical path independently of the worst-case processing time of any instruction

## ■ Bread and butter (common case) design

- ❑ Can optimize the number of states it takes to execute “important” instructions that make up much of the execution time

## ■ Balanced design

- ❑ No need to provide more capability or resources than really needed
  - An instruction that needs resource X multiple times does **not** require multiple X's to be implemented
  - Leads to more efficient hardware: Can reuse hardware components needed multiple times for an instruction

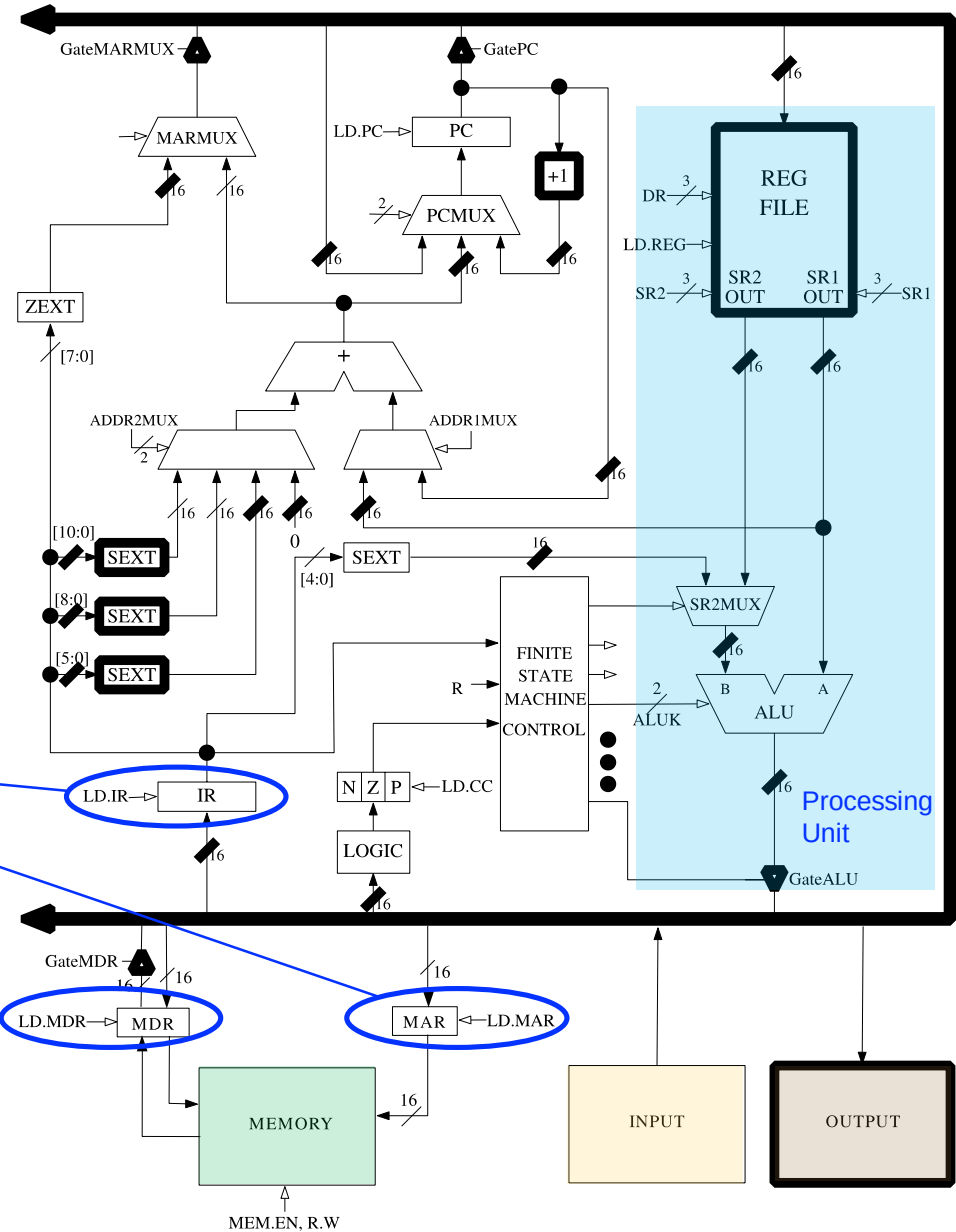
# Downsides of Multi-Cycle Design

---

- **Need to store the intermediate results** at the end of each clock cycle
  - ❑ Hardware overhead for microarchitectural registers
  - ❑ Register setup/hold overhead (i.e., sequencing overhead) is paid multiple times for an instruction
- **Limited concurrency**
  - ❑ Only a small part of the machine is used at any point in time

# Multi-Cycle LC-3 Data Path

Extra registers  
not needed in a  
single-cycle  
design



# Remember: Performance Analysis

---

- Execution time of a single instruction
  - **{CPI} x {clock cycle time}**      CPI: Cycles Per Instruction
- Execution time of an entire program
  - Sum over all instructions [**{CPI} x {clock cycle time}**]
  - **{# of instructions} x {Average CPI} x {clock cycle time}**
- Single-cycle microarchitecture performance
  - CPI = 1
  - Clock cycle time = long
- Multi-cycle microarchitecture performance
  - CPI = different for each instruction
    - Average CPI → hopefully small
  - Clock cycle time = short

**In multi-cycle, we have two degrees of freedom to optimize independently**